

Limpieza y preparación de datos

Adriana Socorro del Pozo, Llara Calvo García, Nadine March Wischniewski, Sergey Sivorilov Kabanov

2026-05-30

Introducción

El presente archivo documenta el proceso de preprocesamiento, limpieza y transformación de los datos. Para la elaboración de este proyecto se ha partido de diversas fuentes de información heterogéneas (registros climáticos, indicadores macroeconómicos, métricas turísticas y calendarios locales). Dada la diferente naturaleza de los datos originales, cada fuente ha sido tratada y depurada de forma individualizada, aplicando las técnicas de limpieza más adecuadas según su contenido específico y sus anomalías particulares.

Fuente de datos 1: Clima

Los datos climáticos fueron obtenidos a través de la API oficial de la Agencia Estatal de Meteorología (AEMET).

Se disponía de 12 datasets diferentes para cada uno de los años de nuestro periodo (2016 a 2025), más otro dataset que incluye los meses y las pernoctaciones totales de cada mes.

En un principio, se valorarán las siguientes variables climáticas de estudio:

- **inso**: media diaria de horas de insolación directa despejada
- **tm_min**: promedio mensual de las temperaturas más bajas registradas diariamente (°C)
- **tm_max**: promedio mensual de las temperaturas más altas alcanzadas cada día (°C)
- **np_010**: número de días del mes en los que se registró una precipitación igual o superior a 1 milímetro.
- **pernoctaciones**: variable respuesta. Número total de noches que los viajeros se han alojado en los establecimientos hoteleros.

1. Cargar los datos y estructura

1.1. Carga

En primer lugar es necesario cargar los 12 datasets independientes descargados de la AEMET mediante su API y juntarlos en un único dataset.

```
datasets_clima <- list.files(pattern = "Clima_Sevilla_.*\\.csv")
df_clima_bruto <- lapply(datasets_clima, read_csv) %>% bind_rows()
```

```
## Rows: 13 Columns: 43
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (36): indicativo, n_cub, glo, hr, n_gra, n_fog, inso, nw_55, q_mar, q_me...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 43
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (36): indicativo, n_cub, glo, hr, n_gra, n_fog, inso, nw_55, q_mar, q_me...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 43
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (36): indicativo, n_cub, glo, hr, n_gra, n_fog, inso, nw_55, q_mar, q_me...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 42
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (35): indicativo, n_cub, glo, hr, n_gra, n_fog, inso, nw_55, q_mar, q_me...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 42
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (35): indicativo, n_cub, glo, hr, n_gra, n_fog, inso, nw_55, q_mar, q_me...
##
```

```
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 42
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (35): indicativo, n_cub, hr, n_gra, n_fog, inso, nw_55, q_mar, q_med, tm...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 42
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (35): indicativo, n_cub, hr, n_gra, n_fog, inso, nw_55, q_mar, q_med, tm...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 13 Columns: 41
## -- Column specification -----
## Delimiter: ","
## chr (7): p_max, q_max, ta_max, w_racha, ta_min, q_min, fecha
## dbl (34): indicativo, n_cub, hr, n_gra, n_fog, inso, nw_55, q_mar, q_med, tm...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Cargo también el dataset de pernoctaciones:

```
df_pernoctaciones <- read_delim("Pernoctaciones_2016_2025_total .csv",
  delim = ";", escape_double = FALSE, trim_ws = TRUE)
```

```
## Rows: 120 Columns: 2
## -- Column specification -----
## Delimiter: ";"
## chr (1): Fecha
## dbl (1): Pernoctaciones_Totales
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

1.2. Estructura de los datos

Vamos a evaluar la estructura de los datos para detectar posibles anomalías en los formatos, tipos de datos incorrectos, etc.

```
# 1. Vemos las primeras filas y la estructura de las variables climáticas
head(df_clima_bruto)
```

```
## # A tibble: 6 x 43
##   indicativo p_max n_cub glo hr n_gra n_fog inso q_max nw_55 q_mar q_med
##   <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <chr> <dbl> <dbl> <dbl>
```

```
## 1      5783 72.5(~      6 40917      63      0      2      7.9 1022~      0 1015. 1012.
## 2      5783 34.4(~      8 28017      72      0      2      6.6 1025~      1 1016. 1013.
## 3      5783 19.2(~      3 25744      79      0      1      7.2 1033~      2 1025. 1021.
## 4      5783 72.5(~     53 627536      58      0     19      9.2 1033~     17 1018. 1014.
## 5      5783 12.0(~      9 24667      81      0      6      5.3 1032~      4 1024. 1020.
## 6      5783 9.4(2~      7 30337      68      0      2      6.2 1029~      3 1022. 1018.
## # i 31 more variables: tm_min <dbl>, ta_max <chr>, ts_min <dbl>, nt_30 <dbl>,
## #   nv_0050 <dbl>, n_des <dbl>, w_racha <chr>, np_100 <dbl>, n_nub <dbl>,
## #   p_sol <dbl>, nw_91 <dbl>, np_001 <dbl>, ta_min <chr>, w_rec <dbl>, e <dbl>,
## #   np_300 <dbl>, nv_1000 <dbl>, evap <dbl>, p_mes <dbl>, n_llu <dbl>,
## #   n_tor <dbl>, w_med <dbl>, nt_00 <dbl>, ti_max <dbl>, n_nie <dbl>,
## #   tm_mes <dbl>, tm_max <dbl>, nv_0100 <dbl>, q_min <chr>, np_010 <dbl>,
## #   fecha <chr>
```

```
str(df_clima_bruto)
```

```
## spc_tbl_ [130 x 43] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ indicativo: num [1:130] 5783 5783 5783 5783 5783 ...
## $ p_max      : chr [1:130] "72.5(22)" "34.4(25)" "19.2(14)" "72.5(22/oct)" ...
## $ n_cub      : num [1:130] 6 8 3 53 9 7 3 8 8 0 ...
## $ glo        : num [1:130] 40917 28017 25744 627536 24667 ...
## $ hr         : num [1:130] 63 72 79 58 81 68 60 62 55 38 ...
## $ n_gra      : num [1:130] 0 0 0 0 0 0 0 0 0 0 ...
## $ n_fog      : num [1:130] 2 2 1 19 6 2 5 0 0 0 ...
## $ inso       : num [1:130] 7.9 6.6 7.2 9.2 5.3 6.2 9.4 8.3 9.7 13.1 ...
## $ q_max      : chr [1:130] "1022.5(28)" "1025.0(09)" "1033.0(23)" "1033.0(23/dic)" ...
## $ nw_55      : num [1:130] 0 1 2 17 4 3 1 2 4 0 ...
## $ q_mar      : num [1:130] 1015 1016 1025 1018 1024 ...
## $ q_med      : num [1:130] 1012 1013 1021 1014 1020 ...
## $ tm_min     : num [1:130] 15.8 9.9 8.4 13.6 9.1 8.1 6.7 11.2 14.1 18.1 ...
## $ ta_max     : chr [1:130] "33.6(04)" "26.9(03)" "21.6(09)" "44.8(05/sep)" ...
## $ ts_min     : num [1:130] 19.2 17.8 13.6 24.9 15.9 15.4 11.5 15.7 17.5 23.6 ...
## $ nt_30      : num [1:130] 8 0 0 127 0 0 0 0 8 24 ...
## $ nv_0050    : num [1:130] 0 0 0 0 0 0 0 0 0 0 ...
## $ n_des      : num [1:130] 9 10 13 158 3 6 12 9 10 16 ...
## $ w_racha    : chr [1:130] "18/12.8(22)" "22/16.9(21)" "07/16.1(18)" "32/24.2(14/feb)" ...
## $ np_100     : num [1:130] 2 4 4 23 2 0 0 3 7 0 ...
## $ n_nub      : num [1:130] 16 12 15 155 19 16 16 13 13 14 ...
## $ p_sol      : num [1:130] 70 65 75 74 53 57 78 63 68 89 ...
## $ nw_91      : num [1:130] 0 0 0 0 0 0 0 0 0 0 ...
## $ np_001     : num [1:130] 6 11 7 81 16 6 9 11 11 0 ...
## $ ta_min     : chr [1:130] "10.6(15)" "4.5(24)" "3.6(31)" "0.3(17/feb)" ...
## $ w_rec      : num [1:130] 214 216 230 246 228 314 215 244 283 274 ...
## $ e          : num [1:130] 159 122 120 131 125 106 94 122 126 133 ...
## $ np_300     : num [1:130] 2 1 0 3 0 0 0 0 0 0 ...
## $ nv_1000    : num [1:130] 0 0 0 2 0 0 2 0 0 0 ...
## $ evap       : num [1:130] 1730 1169 999 23337 604 ...
## $ p_mes      : num [1:130] 119.6 105.3 64.8 619.2 50.4 ...
## $ n_llu      : num [1:130] 7 11 7 84 17 7 8 11 12 0 ...
## $ n_tor      : num [1:130] 1 3 0 13 1 0 2 1 2 0 ...
## $ w_med      : num [1:130] 9 9 11 12 10 15 11 12 13 13 ...
## $ nt_00      : num [1:130] 0 0 0 0 0 0 0 0 0 0 ...
## $ ti_max     : num [1:130] 19.9 14.7 13.6 13.2 13.9 13.2 15.9 16.1 17.3 26.6 ...
## $ n_nie      : num [1:130] 0 0 0 0 0 0 0 0 0 0 ...
```

```

## $ tm_mes      : num [1:130] 21.6 14.7 13 19.8 13.1 13.1 13.4 16.8 20 26 ...
## $ tm_max      : num [1:130] 27.4 19.5 17.5 25.9 17.2 18.1 20.1 22.4 26 33.9 ...
## $ nv_0100     : num [1:130] 0 0 0 0 0 0 0 0 0 0 ...
## $ q_min       : chr [1:130] "998.4(13)" "998.4(22)" "1002.0(03)" "994.8(08/may)" ...
## $ np_010      : num [1:130] 5 8 7 57 9 4 5 7 10 0 ...
## $ fecha       : chr [1:130] "2016-10" "2016-11" "2016-12" "2016-13" ...
## - attr(*, "spec")=
## .. cols(
## ..   indicativo = col_double(),
## ..   p_max = col_character(),
## ..   n_cub = col_double(),
## ..   glo = col_double(),
## ..   hr = col_double(),
## ..   n_gra = col_double(),
## ..   n_fog = col_double(),
## ..   inso = col_double(),
## ..   q_max = col_character(),
## ..   nw_55 = col_double(),
## ..   q_mar = col_double(),
## ..   q_med = col_double(),
## ..   tm_min = col_double(),
## ..   ta_max = col_character(),
## ..   ts_min = col_double(),
## ..   nt_30 = col_double(),
## ..   nv_0050 = col_double(),
## ..   n_des = col_double(),
## ..   w_racha = col_character(),
## ..   np_100 = col_double(),
## ..   n_nub = col_double(),
## ..   p_sol = col_double(),
## ..   nw_91 = col_double(),
## ..   np_001 = col_double(),
## ..   ta_min = col_character(),
## ..   w_rec = col_double(),
## ..   e = col_double(),
## ..   np_300 = col_double(),
## ..   nv_1000 = col_double(),
## ..   evap = col_double(),
## ..   p_mes = col_double(),
## ..   n_llu = col_double(),
## ..   n_tor = col_double(),
## ..   w_med = col_double(),
## ..   nt_00 = col_double(),
## ..   ti_max = col_double(),
## ..   n_nie = col_double(),
## ..   tm_mes = col_double(),
## ..   tm_max = col_double(),
## ..   nv_0100 = col_double(),
## ..   q_min = col_character(),
## ..   np_010 = col_double(),
## ..   fecha = col_character()
## .. )
## - attr(*, "problems")=<externalptr>

```

```
# 2. Vemos las primeras filas y la estructura de la variable pernoctaciones
head(df_pernoctaciones)
```

```
## # A tibble: 6 x 2
##   Fecha      Pernoctaciones_Totales
##   <chr>          <dbl>
## 1 2025-12          522493
## 2 2025-11          560063
## 3 2025-10          628123
## 4 2025-09          571732
## 5 2025-08          523211
## 6 2025-07          487984
```

```
str(df_pernoctaciones)
```

```
## spc_tbl_ [120 x 2] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ Fecha      : chr [1:120] "2025-12" "2025-11" "2025-10" "2025-09" ...
## $ Pernoctaciones_Totales: num [1:120] 522493 560063 628123 571732 523211 ...
## - attr(*, "spec")=
## .. cols(
## ..   Fecha = col_character(),
## ..   Pernoctaciones_Totales = col_double()
## .. )
## - attr(*, "problems")=<externalptr>
```

En el dataframe `df_clima_bruto` se observa que en todos los años hay una fila resumen “mes 13” (por ejemplo, 2025-13) que no nos sirve para el estudio y que es necesario eliminar.

Además, el formato de fecha en los dos dataframes es diferente. En `df_pernoctaciones` aparece en el formato correcto (2025-01), mientras que en `df_clima_bruto` aparece sin el 0 (2025-1).

Vamos a modificar el dataframe `df_clima_bruto` para eliminar estas filas resumen del “mes 13”, dar el formato correcto a la fecha y seleccionar únicamente las variables que nos interesan:

```
df_clima_limpio <- df_clima_bruto %>%
  filter(!str_detect(fecha, "-13$")) %>% # Elimino las filas "mes 13"
  select(fecha, tm_max, tm_min, inso, dias_lluvia = np_010) %>%
  mutate(fecha = format(ymd(paste0(fecha, "-1")), "%Y-%m")) # Modifico la fecha
```

También va a ser necesario modificar los nombres de las columnas del dataframe `df_pernoctaciones` para que estén en minúsculas y no haya problema a la hora de hacer el join futuro.

```
names(df_pernoctaciones) <- tolower(names(df_pernoctaciones))
```

1.3. Unión de dataframes

Una vez que tenemos los dataframes limpios y con los mismos formatos podemos unirlos usando join:

```
df_maestro_clima <- df_clima_limpio %>%
  inner_join(df_pernoctaciones, by = "fecha")
```

1.4. Resumen estadístico

Revisamos de nuevo la estructura de este nuevo dataframe y hacemos un resumen estadístico de todas las variables:

```
head(df_maestro_clima)
```

```
## # A tibble: 6 x 6
##   fecha    tm_max tm_min  inso dias_lluvia pernoctaciones_totales
##   <chr>    <dbl> <dbl> <dbl>    <dbl>          <dbl>
## 1 2016-10   27.4   15.8   7.9         5          507455
## 2 2016-11   19.5    9.9   6.6         8          380143
## 3 2016-12   17.5    8.4   7.2         7          356295
## 4 2016-01   17.2    9.1   5.3         9          303804
## 5 2016-02   18.1    8.1   6.2         4          354605
## 6 2016-03   20.1    6.7   9.4         5          437334
```

```
str(df_maestro_clima)
```

```
## tibble [120 x 6] (S3: tbl_df/tbl/data.frame)
##  $ fecha           : chr [1:120] "2016-10" "2016-11" "2016-12" "2016-01" ...
##  $ tm_max          : num [1:120] 27.4 19.5 17.5 17.2 18.1 20.1 22.4 26 33.9 37.7 ...
##  $ tm_min          : num [1:120] 15.8 9.9 8.4 9.1 8.1 6.7 11.2 14.1 18.1 21.8 ...
##  $ inso            : num [1:120] 7.9 6.6 7.2 5.3 6.2 9.4 8.3 9.7 13.1 12.8 ...
##  $ dias_lluvia     : num [1:120] 5 8 7 9 4 5 7 10 0 0 ...
##  $ pernoctaciones_totales: num [1:120] 507455 380143 356295 303804 354605 ...
```

```
summary(df_maestro_clima)
```

```
##      fecha           tm_max           tm_min           inso
## Length:120      Min.    :15.0      Min.    : 4.0      Min.    : 4.500
## Class :character 1st Qu.:19.0      1st Qu.: 9.1      1st Qu.: 7.150
## Mode  :character Median :25.7      Median :12.8      Median : 9.000
##              Mean  :26.3      Mean  :13.4      Mean  : 9.081
##              3rd Qu.:32.6      3rd Qu.:18.0      3rd Qu.:11.100
##              Max.   :39.4      Max.   :22.8      Max.   :13.300
##              NA's   :3        NA's   :3        NA's   :17
##   dias_lluvia  pernoctaciones_totales
## Min.   : 0.000  Min.   :    0
## 1st Qu.: 1.000  1st Qu.:383467
## Median : 3.000  Median :460734
## Mean   : 3.975  Mean   :424053
## 3rd Qu.: 6.000  3rd Qu.:522557
## Max.   :18.000  Max.   :631726
##
```

Con este resumen estadístico y viendo la estructura del dataframe `df_maestro_clima` se detectan los siguientes puntos que será necesario corregir:

- NA: se observa la presencia de varios valores NA en las variables `tm_max`, `tm_min` y `inso`. El tratamiento de los NA se desarrollará en el siguiente apartado.

2. Evaluación de la calidad de los datos

2.1. Valores Faltantes (NAs)

Como se comentó en el apartado anterior, se detectan valores NA en tres de las variables climáticas seleccionadas. A continuación vamos a sumar el total de NA presentes en `df_maestro_clima` para cada variable:

```
colSums(is.na(df_maestro_clima))
```

```
##           fecha           tm_max           tm_min
##           0             3             3
##        inso      dias_lluvia pernoctaciones_totales
##        17             0             0
```

- `tm_max` presenta 3 valores NA
- `tm_min` presenta 3 valores NA
- `inso` presenta 17 valores NA

Es destacable el caso de los 17 valores NA de la variable `inso`, y prestando más atención a los datos se observa la ausencia de casi todos los valores correspondientes al año 2025.

Esta falta de valores puede deberse a fallos en los sensores de medición, falta de mantenimiento y calibración o errores en la transmisión de los datos entre las estaciones climáticas y los receptores.

2.1.1. Tratamiento de los valores NA En el contexto de modelos predictivos, la eliminación directa de registros con valores faltantes (NA) debe considerarse siempre como último recurso, ya que provoca la pérdida de información en el resto de variables del dataset para esos meses. Por lo tanto, se decide aplicar técnicas de imputación adaptadas a la naturaleza y magnitud de cada ausencia de datos:

- **Valores aislados de `tm_max`, `tm_min` y `inso`:** en el caso de los valores nulos dispersos y de corta duración en las variables de temperatura máxima (`tm_max`, 3 registros), temperatura mínima (`tm_min`, 3 registros) y horas de sol (`inso`, 6 registros) se ha aplicado una interpolación lineal con un hueco máximo de 2 NA seguidos (`maxgap = 2`), estimando el dato faltante a partir de la tendencia natural trazada entre el mes anterior y el posterior al fallo del sensor.

```
df_maestro_clima <- df_maestro_clima %>%
  mutate(tm_max = na.approx(tm_max, na.rm = FALSE, maxgap = 2),
         tm_min = na.approx(tm_min, na.rm = FALSE, maxgap = 2),
         inso    = na.approx(inso, na.rm = FALSE, maxgap = 2))
```

- **Ausencia de valores `inso` para valores faltantes de 2025:** para este caso se opta por la técnica de Imputación Estacional, a través de la cual los valores son reemplazados por la media histórica mensual del periodo disponible (2016-2024). De esta forma se mantiene la estacionalidad relacionada con esta variable por la cual los meses de invierno tienen un valor `inso` más bajo que los meses de verano. Se descarta la medida de Interpolación Lineal seleccionada para los valores aislados porque resultaría ineficaz y distorsionaría la curva climática al trazar una recta anual.

```
df_maestro_clima <- df_maestro_clima %>%
  mutate(mes_auxiliar = month(ymd(paste0(fecha, "-01")))) %>%
  group_by(mes_auxiliar) %>%
  mutate(inso = ifelse(is.na(inso), mean(inso, na.rm = TRUE), inso)) %>%
  ungroup() %>%
  select(-mes_auxiliar)
```


Revisamos la presentación de NA tras aplicar las transformaciones:

```
colSums(is.na(df_maestro_clima))
```

```
##           fecha           tm_max           tm_min
##           0             0             0
##           inso      dias_lluvia pernoctaciones_totales
##           0             0             0
```

2.2. Valores Anómalos (outliers)

No se detectan valores anómalos.

2.3. Valores No válidos:

No se detectan valores no válidos.

2.4. Casos Duplicados

Al tratarse de variables climáticas, la opción de que haya casos duplicados es totalmente factible. La única variable que no debería repetirse es la fecha. Comprobamos que no hay duplicados en esta variable:

```
any(duplicated(df_maestro_clima$fecha))
```

```
## [1] FALSE
```

Fuente de datos 2: viajeros, grado de ocupación y llegadas al aeropuerto de Sevilla

1. Cargar los datos y estructura

1.1. Viajeros

Los datos turísticos fueron obtenidos del Instituto Nacional de Estadística (INE), concretamente de la operación estadística:

Hoteles: encuesta de ocupación, índice de precios e indicadores de rentabilidad

Dentro de este proceso se descargó el conjunto de datos:

- **Viajeros y pernoctaciones por puntos turísticos**

El dataset original contenía información mensual del punto turístico de Sevilla, incluyendo tanto viajeros como pernoctaciones diferenciados por tipo de residencia (residentes en España y residentes en el extranjero).

Inicialmente, los datos se encontraban en formato longitudinal, donde las variables de viajeros y pernoctaciones estaban agrupadas en una misma columna categórica denominada “*Viajeros y pernoctaciones*”. Asimismo, los valores correspondientes a turistas nacionales e internacionales se diferenciaban mediante la variable “*Residencia*”.

Para poder utilizar esta información en el análisis y en los modelos predictivos posteriores, fue necesario realizar un proceso de limpieza, transformación y estructuración de los datos con **Python en Jupyter Notebook** (ver anexo ine_viajeros_limpio.ipynb).

En primer lugar, se importó el archivo original descargado desde el INE, especificando correctamente el separador utilizado y la codificación de caracteres, ya que el dataset contenía símbolos especiales y separadores de miles.

Posteriormente, se realizó la limpieza de la variable numérica principal (**Total**), eliminando los puntos utilizados como separadores de miles y transformando los valores vacíos o representados mediante símbolos especiales en valores nulos compatibles con el tratamiento estadístico.

Una vez limpiados los datos, el dataset fue transformado desde formato longitudinal a formato ancho mediante una tabla pivote. Este proceso permitió separar las diferentes métricas turísticas en variables independientes, obteniendo así:

- viajeros nacionales
- viajeros internacionales
- pernoctaciones nacionales
- pernoctaciones internacionales

A continuación, se generaron nuevas variables agregadas calculando el total de viajeros y el total de pernoctaciones a partir de la suma de turistas nacionales e internacionales.

También se realizó una transformación de la variable temporal. El periodo original se encontraba en formato YYYYMmm (por ejemplo, 2025M12), por lo que se convirtió al formato estándar YYYY-MM para facilitar el análisis cronológico y la integración con el resto de datasets.

Finalmente, el conjunto de datos limpio y estructurado fue exportado a un nuevo archivo CSV (**ine_viajeros_clean.csv**) preparado para su posterior integración en el dataset maestro utilizado en el análisis turístico y en los modelos de predicción.

1.2. Grado de ocupación

El indicador de **grado de ocupación hotelera** fue obtenido del Instituto Nacional de Estadística (INE), concretamente de la operación estadística:

Establecimientos, plazas estimadas, grados de ocupación y personal empleado por puntos turísticos

Este conjunto de datos complementa la información de viajeros y pernoctaciones, ya que permite medir el nivel de saturación hotelera en cada periodo, siendo una variable clave para el análisis de la demanda turística.

El dataset original presentaba ciertos problemas de estructura y consistencia, ya que los valores numéricos incluían distintos formatos (comas y puntos como separadores decimales), además de inconsistencias en la codificación del archivo. Por ello, fue necesario realizar un proceso de limpieza previo a su integración.

En primer lugar, se llevó a cabo la lectura del archivo bruto con **Python en Jupyter Notebook** (ver anexo ine_viajeros_ocupacion_clean.ipynb), adaptando el separador según el formato del fichero. Posteriormente, se corrigieron los valores numéricos de la variable *Total*, unificando el formato decimal y eliminando caracteres no válidos o registros vacíos que podían afectar al análisis.

A continuación, la variable temporal *Periodo* fue transformada al formato estándar YYYY-MM, sustituyendo el formato original del INE (YYYYMmm) para facilitar la integración con el resto de datasets del proyecto.

Una vez depurado el dataset, se conservó únicamente la variable temporal y el indicador de grado de ocupación, renombrando la variable *Total* como **grado_ocupacion** para mejorar su interpretación.

Posteriormente, este dataset fue integrado con el conjunto de datos previamente construido a partir de viajeros y pernoctaciones. Durante este proceso, se eliminaron posibles duplicados o versiones anteriores del indicador de ocupación para garantizar la consistencia del dataset final.

La unión de ambos datasets se realizó mediante la variable temporal *Periodo*, asegurando la correcta alineación de las observaciones mensuales.

Finalmente, el dataset resultante fue reorganizado siguiendo una estructura coherente de variables turísticas, priorizando primero las variables agregadas (pernoctaciones, viajeros y grado de ocupación) y posteriormente las desagregadas por origen de residencia.

El resultado final fue exportado como un único dataset consolidado (**ine_viajeros_ocupacion_clean.csv**), que integra la evolución mensual del turismo en Sevilla y sirve como base para el análisis descriptivo y la modelización posterior.

1.3 Llegadas al aeropuerto de Sevilla

Los datos de llegadas al Aeropuerto de Sevilla fueron obtenidos a través de la plataforma oficial de AENA que requiere registro.

En el apartado “**Consultas personalizadas → Pasajeros/Mercancías por Compañía**”, se descargaron los datos de pasajeros internacionales que llegan al Aeropuerto de Sevilla.

Para la extracción se aplicaron los siguientes filtros:

- Aeropuerto base: Sevilla
- Movimiento: Llegadas
- Tipo de tráfico: Comercial

La estructura del dataset original se configuró en formato matricial, donde:

- Las filas representaban países de origen.
- Las columnas representaban meses.
- Los valores correspondían al número de pasajeros.

Este formato implicaba una alta complejidad para el análisis, ya que los datos no estaban estructurados

****Problemas del dataset original****

El dataset descargado presentaba varias dificultades:

- Estructura en formato “wide” (países $\times$ meses), poco adecuada para análisis temporal.
- Valores numéricos con separadores de miles y formato inconsistente.
- Necesidad de reconstruir series históricas completas por tramos de años.
- Descarga fragmentada debido a las limitaciones de la plataforma de AENA.

****Proceso de transformación****

Para resolver estos problemas, se llevó a cabo un proceso de ingeniería de datos consistente en:

- Lectura del archivo en formato texto plano debido a la estructura irregular del CSV.
- Identificación y extracción de los meses asociados a cada columna.
- Transformación de los datos desde formato matricial (país × mes) a formato longitudinal mensual.
- Agregación de los datos por país y categoría geográfica.
- Cálculo de variables derivadas:
 - Llegadas nacionales (España)
 - Llegadas internacionales totales
 - Desagregación por países relevantes (Francia, Italia, Reino Unido, Países Bajos, Alemania)
 - Cálculo del resto del mundo como categoría agregada
- Generación de una serie temporal mensual coherente.

****Construcción del histórico completo****

Dado que AENA solo permite descargas parciales de dos años, el proceso se repitió para distintos periodos temporales (2016–2017, 2018–2019, 2020–2021, 2022–2023, 2024–2025).

Cada bloque fue procesado de forma independiente y posteriormente:

- Se unificaron todos los datasets en un único histórico.
- Se ordenaron cronológicamente para asegurar la continuidad temporal.
- Se verificó la consistencia de las variables entre periodos.

En total, este proceso se implementó mediante **7 notebooks de Python**, correspondientes a cada bloque temporal de construcción del dataset:

- llegadas_2017_2016.ipynb
- llegadas_2019_2018.ipynb
- llegadas_2021_2020.ipynb
- llegadas_2023_2022.ipynb
- llegadas_2025_2024.ipynb
- llegadas_completo.ipynb
- historico_llegadas_2016_2025.csv

Durante la fase de análisis exploratorio de datos (EDA), se detectaron inconsistencias en el bloque 2022–2023. Tras la revisión, se realizó una nueva descarga de datos oficiales y se generó un archivo corregido:

- 2023-2022_nuevo.csv

Este dataset reemplazó al original y fue integrado nuevamente en el flujo de procesamiento.

Posteriormente, se desarrolló un script específico de actualización para garantizar la correcta integración del bloque corregido dentro del dataset maestro:

Dataset_actualizacion_2023_2022.ipynb**2.4 Integración dataset del bloque viajeros, grado de ocupación y llegadas**

Una vez contruidos y depurados todos los subconjuntos de datos procedentes del INE y de AENA, se realizó la fase final de integración con el objetivo de unificar todas las variables en un único dataset (**Dataset_Nadine_2016_2025.csv**) en Python (ver **anexo Dataset_Nadine_def.ipynb**).

El dataset final resultante tiene una estructura mensual ('Periodo') y combina información de demanda turística, ocupación hotelera y conectividad aérea, permitiendo un análisis conjunto del comportamiento turístico en Sevilla.

El proceso de construcción del dataset final se basa en una **fusión de dos fuentes principales**:

Se utilizó el dataset previamente construido ('historico_llegadas_2025_2016.csv'), que contiene:

- Llegadas totales al aeropuerto
- Llegadas nacionales
- Llegadas internacionales
- Desagregación por países (Francia, Italia, Reino Unido, Países Bajos, Alemania)
- Resto del mundo

Este dataset ya había sido previamente transformado desde el formato original de AENA (matricial país × mes) a formato temporal mensual estándar.

El segundo dataset corresponde a las variables turísticas del INE, incluyendo:

- Pernoctaciones totales
- Viajeros totales
- Grado de ocupación hotelera
- Viajeros nacionales e internacionales
- Pernoctaciones nacionales e internacionales

La integración de ambos conjuntos se realizó mediante los siguientes pasos:

Ambos datasets fueron alineados utilizando la variable:

- 'Mes' en el dataset de AENA
- 'Periodo' en el dataset del INE

Se normalizó el formato a una estructura común 'YYYY-MM' para garantizar la correcta correspondencia temporal.

Se realizó una unión externa (outer join) entre ambos conjuntos:

- Esto permite conservar todos los meses disponibles en ambas fuentes

- Evita la pérdida de información cuando una fuente no tiene datos en un periodo concreto

El dataset resultante se ordenó de forma descendente, desde 2025 hasta 2016, garantizando una estructura cronológica homogénea para el análisis.

El resultado final se exportó como:

- ****Dataset_Nadine_2016_2025.csv****

El dataset consolidado contiene las siguientes variables:

2. Variables finales

2.1. Variables turísticas (INE)

- pernoctaciones_totales
- viajeros_totales
- grado_ocupacion
- viajeros_nacionales
- viajeros_internacionales
- pernoctaciones_nacionales
- pernoctaciones_internacionales

2.2. Variables de conectividad aérea (AENA)

- Volumen total llegadas aeropuerto
- Volumen llegadas Nacionales
- Volumen llegadas Internacionales
- Volumen llegadas Francia
- Volumen llegadas Italia
- Volumen llegadas Reino Unido
- Volumen llegadas Países Bajos
- Volumen llegadas Alemania
- Volumen llegadas Resto del Mundo

En base de dataset trimestral, aplicando el metodo de Denton-Cholette (un metodo estadístico y econometrico que permite desagregación temporal de datos - de baja frecuencia (anuales) a datos de alta frecuencia (trimestrales, mensuales)) para tener datos de GDP mensuales. Tenemos aplicar la misma técnica para cada país y luego hacer un join de datasets.

```
#Despues de obtener datos trimestrales de GDP vamos aplicar method de Denton-Cholette para extrapolear

# 1. Filtrar los datos de Alemania del dataset de 183 registros
datos_de <- pib_trimestral %>%
  filter(geo == "DE") %>%
  arrange(time) # Nos aseguramos de que estén ordenados cronológicamente

# 2. Crear la serie temporal trimestral correcta (Frecuencia 4 = Trimestral)
pib_de_ts <- ts(datos_de$values, start = c(2016, 1), frequency = 4)

# 3. Añadimos 'to = "monthly"' para obligar la desagregación a meses
modelo_mensual <- td(pib_de_ts ~ 1, to = "monthly", conversion = "sum", method = "denton-cholette")

# 4. Extraer el vector de predicción
pib_mensual_vector <- as.numeric(predict(modelo_mensual))

# 5. Crear el desglose de fechas mensuales adaptado a la longitud REAL
fechas_mensuales <- seq(as.Date("2016-01-01"), by = "month", length.out = length(pib_mensual_vector))

# 6. Crear la tabla final estructurada
tabla_mensual_de <- data.frame(
  Fecha = fechas_mensuales,
  Pais = "Alemania",
  PIB_Mensual = pib_mensual_vector
)

# ---- Verificaciones ----
print(paste("Cantidad de trimestres originales de DE:", nrow(datos_de)))
```

```
## [1] "Cantidad de trimestres originales de DE: 41"
```

```
print(paste("Cantidad de meses generados:", nrow(tabla_mensual_de)))
```

```
## [1] "Cantidad de meses generados: 123"
```

La misma operacion para Francia:

```
# 1. Filtrar los datos de Francia (FR)
datos_fr <- pib_trimestral %>%
  filter(geo == "FR") %>%
  arrange(time)

# 2. Crear la serie temporal trimestral para Francia
pib_fr_ts <- ts(datos_fr$values, start = c(2016, 1), frequency = 4)

# 3. Aplicar el modelo Denton-Cholette con 'to = "monthly"'
modelo_mensual_fr <- td(pib_fr_ts ~ 1, to = "monthly", conversion = "sum", method = "denton-cholette")
```



```

pib_mensual_vector_fr <- as.numeric(predict(modelo_mensual_fr))

# 4. Crear las fechas mensuales reales
fechas_mensuales_fr <- seq(as.Date("2016-01-01"), by = "month", length.out = length(pib_mensual_vector_fr))

# 5. Crear la tabla final estructurada para Francia
tabla_mensual_fr <- data.frame(
  Fecha = fechas_mensuales_fr,
  Pais = "Francia",
  PIB_Mensual = pib_mensual_vector_fr
)

# ---- Verificación ----
print(paste("Cantidad de meses generados para Francia:", nrow(tabla_mensual_fr)))

```

```
## [1] "Cantidad de meses generados para Francia: 123"
```

La misma operación para Reino Unido. Como podemos ver con los datos de Reino Unido tenemos 57 meses generados por el tema de disponibilidad de datos después de Brexit. La ausencia de datos se tratará más adelante.

```

# 1. Filtrar los datos del Reino Unido (UK)
datos_uk <- pib_trimestral %>%
  filter(geo == "UK") %>%
  arrange(time)

# 2. Crear la serie temporal trimestral para el Reino Unido
# Mantenemos start = c(2016, 1) y frecuencia trimestral
pib_uk_ts <- ts(datos_uk$values, start = c(2016, 1), frequency = 4)

# 3. Aplicar el modelo Denton-Cholette con 'to = "monthly"'
modelo_mensual_uk <- td(pib_uk_ts ~ 1, to = "monthly", conversion = "sum", method = "denton-cholette")
pib_mensual_vector_uk <- as.numeric(predict(modelo_mensual_uk))

# 4. Crear las fechas mensuales reales
# Usamos length.out dinámico porque la serie del Reino Unido
fechas_mensuales_uk <- seq(as.Date("2016-01-01"), by = "month", length.out = length(pib_mensual_vector_uk))

# 5. Crear la tabla final estructurada para el Reino Unido
tabla_mensual_uk <- data.frame(
  Fecha = fechas_mensuales_uk,
  Pais = "Reino Unido",
  PIB_Mensual = pib_mensual_vector_uk
)

# ---- Verificación de control en consola ----
print(paste("Trimestres disponibles en Eurostat para UK:", nrow(datos_uk)))

```

```
## [1] "Trimestres disponibles en Eurostat para UK: 19"
```

```
print(paste("Meses generados para Reino Unido:", nrow(tabla_mensual_uk)))
```

```
## [1] "Meses generados para Reino Unido: 57"
```

Repetimos la operacion para Paises Bajos.

```
# 1. Filtrar los datos del Paises Bajos (NL)
datos_nl <- pib_trimestral %>%
  filter(geo == "NL") %>%
  arrange(time)

# 2. Crear la serie temporal trimestral para el NL
# Mantenemos start = c(2016, 1) y frecuencia trimestral
pib_nl_ts <- ts(datos_nl$values, start = c(2016, 1), frequency = 4)

# 3. Aplicar el modelo Denton-Cholette con 'to = "monthly"'
modelo_mensual_nl <- td(pib_nl_ts ~ 1, to = "monthly", conversion = "sum", method = "denton-cholette")
pib_mensual_vector_nl <- as.numeric(predict(modelo_mensual_nl))

# 4. Crear las fechas mensuales reales
# Usamos length.out dinámico porque la serie NL
fechas_mensuales_nl <- seq(as.Date("2016-01-01"), by = "month", length.out = length(pib_mensual_vector_nl))

# 5. Crear la tabla final estructurada para el Reino Unido
tabla_mensual_nl <- data.frame(
  Fecha = fechas_mensuales_nl,
  Pais = "Paises Bajos",
  PIB_Mensual = pib_mensual_vector_nl
)

# ---- Verificación de control en consola ----
print(paste("Trimestres disponibles en Eurostat para Paises Bajos:", nrow(datos_nl)))
```

```
## [1] "Trimestres disponibles en Eurostat para Paises Bajos: 41"
```

```
print(paste("Meses generados para Paises Bajos:", nrow(tabla_mensual_nl)))
```

```
## [1] "Meses generados para Paises Bajos: 123"
```

Y finalmente repetimos la operacion para Suecia

```
# 1. Filtrar los datos del Suecia (SE)
datos_se <- pib_trimestral %>%
  filter(geo == "SE") %>%
  arrange(time)

# 2. Crear la serie temporal trimestral para el Suecia
# Mantenemos start = c(2016, 1) y frecuencia trimestral
pib_se_ts <- ts(datos_se$values, start = c(2016, 1), frequency = 4)

# 3. Aplicar el modelo Denton-Cholette con 'to = "monthly"'
```

```

modelo_mensual_se <- td(pib_se_ts ~ 1, to = "monthly", conversion = "sum", method = "denton-cholette")
pib_mensual_vector_se <- as.numeric(predict(modelo_mensual_se))

# 4. Crear las fechas mensuales reales
# Usamos length.out dinámico porque la serie
fechas_mensuales_se <- seq(as.Date("2016-01-01"), by = "month", length.out = length(pib_mensual_vector_se))

# 5. Crear la tabla final estructurada para el Reino Unido
tabla_mensual_se <- data.frame(
  Fecha = fechas_mensuales_se,
  Pais = "Suecia",
  PIB_Mensual = pib_mensual_vector_se
)

# ---- Verificación de control en consola ----
print(paste("Trimestres disponibles en Eurostat para Suecia:", nrow(datos_se)))

```

```
## [1] "Trimestres disponibles en Eurostat para Suecia: 41"
```

```
print(paste("Meses generados para Suecia:", nrow(tabla_mensual_se)))
```

```
## [1] "Meses generados para Suecia: 123"
```

En la siguiente etapa hacemos el join para obtener el primer dataset - dinamica mensual de PIB de países principal emisoras de turismo a Sevilla. El dataset resultante contine 66 NA por falta de datos Reino Unido.

```

# Hacemos el dataset unico para GDP de todos los países

# 1. Renombrar la columna del PIB en cada tabla
df_de <- tabla_mensual_de %>% select(Fecha, DE = PIB_Mensual)
df_fr <- tabla_mensual_fr %>% select(Fecha, FR = PIB_Mensual)
df_se <- tabla_mensual_se %>% select(Fecha, SE = PIB_Mensual)
df_nl <- tabla_mensual_nl %>% select(Fecha, NL = PIB_Mensual)
df_uk <- tabla_mensual_uk %>% select(Fecha, UK = PIB_Mensual)

# 2. Usar left_join para mantener las 123 fechas de base
dataset_gdp_join <- df_de %>%
  left_join(df_fr, by = "Fecha") %>%
  left_join(df_se, by = "Fecha") %>%
  left_join(df_nl, by = "Fecha") %>%
  left_join(df_uk, by = "Fecha") # UK se acoplará hasta donde tenga datos, el resto será NA

# Ver el resultado definitivo
print(paste("Total de filas en el dataset final:", nrow(dataset_gdp_join)))

```

```
## [1] "Total de filas en el dataset final: 123"
```

```
print("Conteo de valores NA por país:")
```

```
## [1] "Conteo de valores NA por país:"
```

```
print(colSums(is.na(dataset_gdp_join)))
```

```
## Fecha      DE      FR      SE      NL      UK
##      0      0      0      0      0      66
```

```
# Ver el resultado
```

```
print(head(dataset_gdp_join))
```

```
##      Fecha      DE      FR      SE      NL      UK
## 1 2016-01-01 261228.1 184829.7 37950.35 58871.16 222554.8
## 2 2016-02-01 261295.0 184764.9 37999.89 58930.54 222705.8
## 3 2016-03-01 261428.8 184635.1 38098.96 59049.30 223007.8
## 4 2016-04-01 261629.5 184440.6 38247.57 59227.43 223460.8
## 5 2016-05-01 261882.2 184344.2 38356.27 59405.84 223801.8
## 6 2016-06-01 262187.0 184346.1 38425.05 59584.53 224031.0
```

```
summary(dataset_gdp_join)
```

```
##      Fecha      DE      FR      SE
## Min.   :2016-01-01   Min.   :243684   Min.   :159910   Min.   :37593
## 1st Qu.:2018-07-16   1st Qu.:271744   1st Qu.:189516   1st Qu.:39787
## Median :2021-02-01   Median :276133   Median :196504   Median :41858
## Mean   :2021-01-30   Mean   :273396   Mean   :195197   Mean   :41395
## 3rd Qu.:2023-08-16   3rd Qu.:277351   3rd Qu.:202741   3rd Qu.:42897
## Max.   :2026-03-01   Max.   :280380   Max.   :207444   Max.   :44311
##
##      NL      UK
## Min.   :57778   Min.   :180248
## 1st Qu.:62344   1st Qu.:224402
## Median :64563   Median :229193
## Mean   :65326   Mean   :226194
## 3rd Qu.:68965   3rd Qu.:232063
## Max.   :70874   Max.   :237752
##
##      NA's      :66
```

```
sum(is.na(dataset_gdp_join))
```

```
## [1] 66
```

```
str(dataset_gdp_join)
```

```
## 'data.frame':   123 obs. of  6 variables:
## $ Fecha: Date, format: "2016-01-01" "2016-02-01" ...
## $ DE : num 261228 261295 261429 261629 261882 ...
## $ FR : num 184830 184765 184635 184441 184344 ...
## $ SE : num 37950 38000 38099 38248 38356 ...
## $ NL : num 58871 58931 59049 59227 59406 ...
## $ UK : num 222555 222706 223008 223461 223802 ...
```

La siguiente etapa es la obtención de los datos de desempleo desde API Eurostat para los principales emisoras del turismo a Sevilla

```
# 1. Descargar y filtrar los datos originales
mis_paises <- c("DE", "FR", "UK", "NL", "SE")
```

```
mi_tabla_paro_total <- get_eurostat(
  "une_rt_m",
  filters = list(
    geo = mis_paises,
    sinceTimePeriod = "2016",
    sex = "T",
    age = "TOTAL",
    unit = "PC_ACT",
    s_adj = "SA"
  ),
  time_format = "date"
)
```

```
## Dataset query already saved in cache_list.json...
```

```
## Reading cache file /var/folders/2f/bkghw3j92112vxw4zwwq8vmhm0000gn/T//RtmpJPjodm/eurostat/67d68394a75
```

```
## Table une_rt_m read from cache file: /var/folders/2f/bkghw3j92112vxw4zwwq8vmhm0000gn/T//RtmpJPjodm
```

```
# 2. SELECCIONAR, RENOMBRAR Y TRADUCIR LOS PAÍSES
```

```
tabla_final <- mi_tabla_paro_total %>%
  select(
    geo,
    time = time,
    paro = values
  ) %>%
  mutate(
    # Reemplazamos los códigos de la columna 'geo' por sus nombres en castellano
    geo = case_match(
      geo,
      "DE" ~ "Alemania",
      "FR" ~ "Francia",
      "UK" ~ "Reino Unido",
      "NL" ~ "Países Bajos",
      "SE" ~ "Suecia",
      .default = geo # Por si acaso se colara otro código, lo deja igual
    )
  )
```

```
## Warning: There was 1 warning in 'mutate()'.
## i In argument: 'geo = case_match(...)'
## Caused by warning:
## ! 'case_match()' was deprecated in dplyr 1.2.0.
## i Please use 'recode_values()' instead.
```

Transformamos la columna 'time' en un objeto de año y mes real. Como se observa el dataset de nuevo contiene un volumen elevado de N/a vinculados con GB.

```

tabla_mes_fecha <- tabla_final %>%
  mutate(

    time = as.yearmon(time)
  )
summary(tabla_mes_fecha)

```

```

##      geo           time      paro
## Length:620      Min.   :2016   Min.   : 2.900
## Class :character 1st Qu.:2019   1st Qu.: 3.800
## Mode  :character Median :2021   Median : 5.300
##              Mean  :2021   Mean  : 5.809
##              3rd Qu.:2024   3rd Qu.: 7.600
##              Max.   :2026   Max.   :10.400
##              NA's    :69

```

Elimina la fila si el valor del paro es NA.

```

tabla_sin_na <- tabla_mes_fecha %>%
  drop_na(paro)

# Comprobación: cuenta cuántos NA quedan ahora
sum(is.na(tabla_sin_na$paro))

```

```
## [1] 0
```

En base de los dos datasets (paro y GDP) hacemos un join por pais y fecha.

```

# 1. Transformar el dataset de GDP a formato "largo" y ajustar la fecha
gdp_largo <- dataset_gdp_join %>%
  # Pasamos de formato ancho a largo
  pivot_longer(
    cols = c(DE, FR, SE, NL, UK),
    names_to = "codigo_pais",
    values_to = "gdp"
  ) %>%
  # Convertimos la fecha a 'yearmon' y recodificamos los códigos a nombres completos
  mutate(
    time = as.yearmon(Fecha),
    geo = case_when(
      codigo_pais == "DE" ~ "Alemania",
      codigo_pais == "FR" ~ "Francia",
      codigo_pais == "UK" ~ "Reino Unido",
      codigo_pais == "NL" ~ "Países Bajos",
      codigo_pais == "SE" ~ "Suecia"
    )
  ) %>%
  # Nos quedamos solo con las columnas necesarias para el join
  select(geo, time, gdp)

colnames(gdp_largo)

```

```
## [1] "geo" "time" "gdp"
```

```
dataset_final <- tabla_sin_na %>%  
  left_join(gdp_largo, by = c("geo", "time"))  
  
head(dataset_final)
```

```
## # A tibble: 6 x 4  
##   geo      time      paro      gdp  
##   <chr>   <yearmon> <dbl>   <dbl>  
## 1 Alemania Jan 2016    4.1 261228.  
## 2 Alemania Feb 2016    4.1 261295.  
## 3 Alemania Mar 2016     4 261429.  
## 4 Alemania Apr 2016     4 261629.  
## 5 Alemania May 2016     4 261882.  
## 6 Alemania Jun 2016     4 262187.
```

```
str(dataset_final)
```

```
## tibble [551 x 4] (S3: tbl_df/tbl/data.frame)  
## $ geo : chr [1:551] "Alemania" "Alemania" "Alemania" "Alemania" ...  
## $ time: 'yearmon' num [1:551] Jan 2016 Feb 2016 Mar 2016 Apr 2016 ...  
## $ paro: num [1:551] 4.1 4.1 4 4 4 4 3.9 3.9 3.8 3.7 ...  
## $ gdp : num [1:551] 261228 261295 261429 261629 261882 ...
```

```
summary(dataset_final)
```

```
##      geo      time      paro      gdp  
## Length:551      Min.    :2016      Min.    : 2.900      Min.    : 37593  
## Class :character 1st Qu.:2018      1st Qu.: 3.800      1st Qu.: 60556  
## Mode  :character Median :2021      Median : 5.300      Median :188786  
##              Mean  :2021      Mean  : 5.809      Mean  :152380  
##              3rd Qu.:2023      3rd Qu.: 7.600      3rd Qu.:232063  
##              Max.   :2026      Max.   :10.400      Max.   :280380  
##              NA's   :2
```

En la siguiente etapa cargamos el dataset de pernoctaciones. Esta será la variable predictiva para nuestro futuro modelo. Como podemos observar el dataset tiene 2 N/a que corresponden al año 2020 y que en próximos pasos vamos a borrar de investigación por ser uno de los dos años pandémicos y que no representan la situación real del turismo.

```
# Leer el archivo de Excel y limpiarlo  
pernoctaciones_limpio <- read_excel(  
  "Pernoctaciones Sevilla_2016-25.xlsx",  
  skip = 9,  
  col_names = c("Fecha", "Residentes_Espana", "Residentes_Extranjero")  
) %>%  
  # Filtrar notas del final y filas vacías  
  filter(str_detect(Fecha, "\\d{4}M\\d{2}")) %>%  
  
  # Forzar que los datos sean numéricos
```

```
mutate(
  Residentes_Espana = as.numeric(Residentes_Espana),
  Residentes_Extranjero = as.numeric(Residentes_Extranjero)
) %>%

# Calcular fecha limpia y la suma total
mutate(
  time = as.yearmon(str_replace(Fecha, "M", "-")),
  total_pernoctaciones = Residentes_Espana + Residentes_Extranjero
) %>%

select(time, total_pernoctaciones)
```

```
## Warning: There were 2 warnings in 'mutate()'.
## The first warning was:
## i In argument: 'Residentes_Espana = as.numeric(Residentes_Espana)'.
## Caused by warning:
## ! NAs introduced by coercion
## i Run 'dplyr::last_dplyr_warnings()' to see the 1 remaining warning.
```

```
# Verificar cómo queda el dataset
head(pernoctaciones_limpio)
```

```
## # A tibble: 6 x 2
##   time      total_pernoctaciones
##   <yearmon>          <dbl>
## 1 Dec 2025          522493
## 2 Nov 2025          560063
## 3 Oct 2025          628123
## 4 Sep 2025          571732
## 5 Aug 2025          523211
## 6 Jul 2025          487984
```

```
str(pernoctaciones_limpio)
```

```
## tibble [120 x 2] (S3: tbl_df/tbl/data.frame)
## $ time          : 'yearmon' num [1:120] Dec 2025 Nov 2025 Oct 2025 Sep 2025 ...
## $ total_pernoctaciones: num [1:120] 522493 560063 628123 571732 523211 ...
```

```
summary(pernoctaciones_limpio)
```

```
##      time      total_pernoctaciones
## Min.   :2016   Min.      :      0
## 1st Qu.:2018   1st Qu.:392790
## Median :2021   Median :464216
## Mean   :2021   Mean   :431250
## 3rd Qu.:2023   3rd Qu.:522686
## Max.   :2026   Max.    :631726
##      NA's      :2
```



```
# Ver cuáles son las filas con NA
pernoctaciones_limpio %>%
  filter(is.na(time) | is.na(total_pernoctaciones))
```

```
## # A tibble: 2 x 2
##   time      total_pernoctaciones
##   <yearmon>          <dbl>
## 1 Jun 2020              NA
## 2 May 2020              NA
```

Se añaden los pernoctaciones al dataset de GDP/paro.

```
# Hacemos el join usando únicamente la columna 'time' como llave
dataset_completo <- dataset_final %>%
  left_join(pernoctaciones_limpio, by = "time")

# Verificar el resultado final
head(dataset_completo)
```

```
## # A tibble: 6 x 5
##   geo      time      paro      gdp total_pernoctaciones
##   <chr>   <yearmon> <dbl>   <dbl>          <dbl>
## 1 Alemania Jan 2016   4.1 261228.      303804
## 2 Alemania Feb 2016   4.1 261295.      354605
## 3 Alemania Mar 2016    4  261429.      437334
## 4 Alemania Apr 2016    4  261629.      461849
## 5 Alemania May 2016    4  261882.      473016
## 6 Alemania Jun 2016    4  262187.      427837
```

En este etapa al dataset actual GDP/paro/pernoctaciones se añaden datos de índice de percepción turística. Es un índice compuesto que incluye varias observaciones: percepción turística global, satisfacción con productos turísticos, percepción de seguridad y percepción climática. Este dataset tiene granularidad mensual y antigüedad de 7 años con desgloses por los países

```
Sys.setlocale("LC_ALL", "es_ES.UTF-8")
```

```
## [1] "es_ES.UTF-8/es_ES.UTF-8/es_ES.UTF-8/C/es_ES.UTF-8/en_US.UTF-8"
```

Primero, descargamos el dataset de indice de satisfacion.

```
index_satisfaction <- read.csv2("ind_satisfaccion_percepcion-Andalucia-Sevilla.csv",
  fileEncoding = "Latin1",
  stringsAsFactors = FALSE)

# Comprobar las primeras filas
head(index_satisfaction)
```

```
##   AÑO MES      PAIS_ORIGEN  CCAA PROVINCIA
## 1 2019   1      Alemania Andalucía  Sevilla
## 2 2019   1      Dinamarca Andalucía  Sevilla
```

```
## 3 2019 1 España Andalucía Sevilla
## 4 2019 1 Estados Unidos de América Andalucía Sevilla
## 5 2019 1 Francia Andalucía Sevilla
## 6 2019 1 Italia Andalucía Sevilla
## INIDICE_PERCEPCION_TURISTICA_GLOBAL INDICE_SATISFACCION_PRODUCTOS_TURISTICOS
## 1 86.94 77.11
## 2 86.95 75.19
## 3 82.88 69.68
## 4 88.43 73.13
## 5 63.37 76.83
## 6 86.67 73.90
## INDICE_PERCEPCION_SEGURIDAD INDICE_PERCEPCION_CLIMATICA
## 1 95.74 87.37
## 2 100.00 100.00
## 3 89.32 94.63
## 4 100.00 100.00
## 5 90.43 78.69
## 6 96.35 90.96
```

Se filtra el dataset de satisfacción dejando solo países de interés. Se aprecia que dataset de satisfacción no tiene datos para Países Bajos, nos impone primer cambio a nuestra investigación - eliminación del países del dataset final por falta completa de uno de los datos. Otra limitacion que se ha encontrado, es que algunas variables solo están disponibles desde 2019, lo que limita el horizonte de modelo.

1. Filtramos el archivo

```
index_satisfaction_filtrados <- index_satisfaction %>%
  filter(PAIS_ORIGEN %in% c("Alemania", "Francia", "Reino Unido", "Suecia", "Países Bajos") |
    grepl("Pa.ses Bajos", PAIS_ORIGEN))
```

2. Comprobamos que solo existan esos países en el nuevo dataframe

```
unique(index_satisfaction_filtrados$PAIS_ORIGEN)
```

```
## [1] "Alemania" "Francia" "Reino Unido" "Suecia"
```

El análisis de estructura de los datos de satisfacción enseña que existen 37 n/a que deben que ser tratados. Además para temas de modelación nos quedamos con solo una columna que demuestra el índice global.

```
summary(index_satisfaction_filtrados)
```

```
##      AÑO      MES      PAIS_ORIGEN      CCAA
## Min.   :2019   Min.    : 1.000   Length:348   Length:348
## 1st Qu.:2020   1st Qu.: 3.000   Class :character   Class :character
## Median :2022   Median : 6.000   Mode  :character   Mode  :character
## Mean    :2022   Mean    : 6.345
## 3rd Qu.:2024   3rd Qu.: 9.000
## Max.    :2026   Max.    :12.000
##
## PROVINCIA      INIDICE_PERCEPCION_TURISTICA_GLOBAL
## Length:348      Min.    : 18.57
## Class :character 1st Qu.: 70.51
## Mode  :character Median : 80.56
```

```
##           Mean    : 78.62
##           3rd Qu.: 87.52
##           Max.    :100.00
##           NA's     :37
## INDICE_SATISFACCION_PRODUCTOS_TURISTICOS INDICE_PERCEPCION_SEGURIDAD
## Min.      : 2.89           Min.      : 40.90
## 1st Qu.:65.53           1st Qu.: 93.53
## Median :70.48           Median :100.00
## Mean     :65.97           Mean    : 95.75
## 3rd Qu.:75.51           3rd Qu.:100.00
## Max.     :91.17           Max.     :100.00
## NA's      :40
## INDICE_PERCEPCION_CLIMATICA
## Min.      : 8.09
## 1st Qu.: 92.28
## Median :100.00
## Mean     : 94.95
## 3rd Qu.:100.00
## Max.     :100.00
## NA's      :2
```

```
index_satisfaction_filtrados_con_na <- index_satisfaction_filtrados %>%
  filter(if_any(everything(), is.na))
```

```
index_satisfaction_filtrados_sin_na <- index_satisfaction_filtrados %>%
  group_by(PAIS_ORIGEN, MES) %>%
  mutate(
    # Reemplaza los NA en cada columna numérica por la media de su grupo (país-mes)
    INIDICE_PERCEPCION_TURISTICA_GLOBAL = ifelse(
      is.na(INIDICE_PERCEPCION_TURISTICA_GLOBAL),
      mean(INIDICE_PERCEPCION_TURISTICA_GLOBAL, na.rm = TRUE),
      INIDICE_PERCEPCION_TURISTICA_GLOBAL
    )
  ) %>%
  ungroup() %>%
  select(
    -INDICE_SATISFACCION_PRODUCTOS_TURISTICOS,
    -INDICE_PERCEPCION_SEGURIDAD,
    -INDICE_PERCEPCION_CLIMATICA
  )

summary(index_satisfaction_filtrados_sin_na)
```

```
##      AÑO      MES      PAIS_ORIGEN      CCAA
## Min.   :2019   Min.    : 1.000   Length:348   Length:348
## 1st Qu.:2020   1st Qu.: 3.000   Class :character   Class :character
## Median :2022   Median : 6.000   Mode  :character   Mode  :character
## Mean   :2022   Mean    : 6.345
## 3rd Qu.:2024   3rd Qu.: 9.000
## Max.   :2026   Max.    :12.000
## PROVINCIA      INIDICE_PERCEPCION_TURISTICA_GLOBAL
## Length:348      Min.      : 18.57
## Class :character 1st Qu.: 70.98
```

```
## Mode :character Median : 80.17
## Mean : 78.71
## 3rd Qu.: 87.30
## Max. :100.00
```

Se realizará un join entre dataset de GDP/paro/pernoctaciones e índice de satisfacción.

```
# =====
# STEP 1: Limpiar el dataset de percepción (Solo las 4 columnas)
# =====
dataset_satisfaction_limpio <- index_satisfaction_filtrados_sin_na %>%
  select(AÑO, MES, INIDICE_PERCEPCION_TURISTICA_GLOBAL, PAIS_ORIGEN)

# =====
# STEP 2: Preparar el 'dataset_completo' descomponiendo 'time'
# =====
# Extraemos el año y el mes de la variable <S3: yearmon> para que coincidan
# con los tipos de datos del dataset de percepción.

dataset_gdp_paro_pernocta_index <- dataset_completo %>%
  mutate(
    AÑO = as.numeric(format(time, "%Y")),
    MES = as.numeric(format(time, "%m"))
  ) %>%
  rename(PAIS_ORIGEN = geo) %>%
  select(-time) # Eliminamos la columna original para no duplicar fechas

# =====
# STEP 3: EL JOIN MENSUAL DEFINITIVO
# =====
# Unimos por las tres claves comunes: AÑO, MES y PAIS_ORIGEN.

dataset_gdp_paro_pernocta_index_final <- dataset_satisfaction_limpio %>%
  left_join(dataset_gdp_paro_pernocta_index, by = c("AÑO", "MES", "PAIS_ORIGEN"))

# =====
# VERIFICACIÓN
# =====
# view(dataset_gdp_paro_pernocta_index_final)
print(head(dataset_gdp_paro_pernocta_index_final, 10))
```

```
## # A tibble: 10 x 7
##   AÑO MES INIDICE_PERCEPCION_TURISTICA_GLOBAL PAIS_ORIGEN paro gdp
##   <dbl> <dbl> <dbl> <chr> <dbl> <dbl>
## 1 2019 1 86.9 Alemania 3 275802.
## 2 2019 1 63.4 Francia 8.8 195065.
## 3 2019 1 87.4 Reino Unido 3.8 233122.
## 4 2019 1 80.0 Suecia 6.3 40372.
## 5 2019 2 76.5 Alemania 3 276228.
## 6 2019 2 78.5 Francia 8.7 195645.
```

```
## 7 2019 2 94.0 Reino Unido 3.7 233513.
## 8 2019 2 88.8 Suecia 6.9 40436.
## 9 2019 3 81.8 Alemania 3 276326.
## 10 2019 3 69.8 Francia 8.7 196118.
## # i 1 more variable: total_pernoctaciones <dbl>
```

En la siguiente etapa se descarga el dataset de precios de hoteles de 3- 5 estrellas desglosado por fecha de antelación de reservas.

```
datos_hoteles <- read_csv2(
  "precios_alojamientos_hoteleros-Andalucia-Sevilla.csv",
  locale = locale(encoding = "ISO-8859-1")
)
```

```
## i Using "','" as decimal and "'.'" as grouping mark. Use 'read_delim()' for more control.
```

```
## Rows: 1044 Columns: 8
## -- Column specification -----
## Delimiter: ";"
## chr (4): CATEGORIA_ALOJAMIENTO, CCAA, PROVINCIA, PERIODO_ANTELACION
## dbl (4): AÑO, MES, PRECIO_CHECK-IN_ENTRE_SEMANA, PRECIO_CHECK-IN_FIN_SEMANA
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
datos_hoteles_filtrados <- datos_hoteles %>%
  select(-CCAA, -PROVINCIA)

head(datos_hoteles_filtrados)
```

```
## # A tibble: 6 x 6
##   AÑO MES CATEGORIA_ALOJAMIENTO PERIODO_ANTELACION PRECIO_CHECK-IN_ENTRE_S~1
##   <dbl> <dbl> <chr> <chr> <dbl>
## 1 2019 1 Hotel 3 Estrellas 1 Mes 58
## 2 2019 1 Hotel 4 Estrellas 1 Mes 82
## 3 2019 1 Hotel 5 Estrellas 1 Mes 210
## 4 2019 1 Hotel 3 Estrellas 1 Semana 59
## 5 2019 1 Hotel 4 Estrellas 1 Semana 85
## 6 2019 1 Hotel 5 Estrellas 1 Semana 197
## # i abbreviated name: 1: 'PRECIO_CHECK-IN_ENTRE_SEMANA'
## # i 1 more variable: 'PRECIO_CHECK-IN_FIN_SEMANA' <dbl>
```

Renombramos los nombres de algunas columnas para evitar problemas con su identificación.

```
summary(datos_hoteles_filtrados)
```

```
##   AÑO      MES      CATEGORIA_ALOJAMIENTO PERIODO_ANTELACION
## Min.   :2019 Min.   : 1.000 Length:1044      Length:1044
## 1st Qu.:2020 1st Qu.: 3.000 Class :character Class :character
## Median :2022 Median : 6.000 Mode  :character Mode  :character
## Mean   :2022 Mean   : 6.345
```

```
## 3rd Qu.:2024    3rd Qu.: 9.000
## Max.    :2026    Max.    :12.000
##
## PRECIO_CHECK-IN_ENTRE_SEMANA PRECIO_CHECK-IN_FIN_SEMANA
## Min.    : 55.0          Min.    : 62.0
## 1st Qu.: 93.0          1st Qu.:112.0
## Median :128.0          Median :156.0
## Mean    :161.7          Mean    :185.8
## 3rd Qu.:217.0          3rd Qu.:240.5
## Max.    :579.0          Max.    :549.0
## NA's    :1             NA's    :9
```

```
str(datos_hoteles_filtrados)
```

```
## tibble [1,044 x 6] (S3: tbl_df/tbl/data.frame)
## $ AÑO          : num [1:1044] 2019 2019 2019 2019 2019 ...
## $ MES          : num [1:1044] 1 1 1 1 1 1 1 1 1 1 ...
## $ CATEGORIA_ALOJAMIENTO : chr [1:1044] "Hotel 3 Estrellas" "Hotel 4 Estrellas" "Hotel 5 Estre..."
## $ PERIODO_ANTELACION   : chr [1:1044] "1 Mes" "1 Mes" "1 Mes" "1 Semana" ...
## $ PRECIO_CHECK-IN_ENTRE_SEMANA: num [1:1044] 58 82 210 59 85 197 58 82 201 57 ...
## $ PRECIO_CHECK-IN_FIN_SEMANA  : num [1:1044] 76 99 228 NA NA NA 81 104 235 70 ...
```

```
# Renombrar para quitar el guion medio y trabajar tranquilos
datos_hoteles_filtrados_final <- datos_hoteles_filtrados %>%
  rename(
    PRECIO_ENTRE_SEMANA = `PRECIO_CHECK-IN_ENTRE_SEMANA`,
    PRECIO_FIN_SEMANA = `PRECIO_CHECK-IN_FIN_SEMANA`
  )

# Ahora ya puedes filtrar normalmente sin errores:
filas_con_na <- datos_hoteles_filtrados_final %>%
  filter(is.na(PRECIO_ENTRE_SEMANA) | is.na(PRECIO_FIN_SEMANA))

head(filas_con_na, 10)
```

```
## # A tibble: 10 x 6
##   AÑO   MES CATEGORIA_ALOJAMIENTO PERIODO_ANTELACION PRECIO_ENTRE_SEMANA
##   <dbl> <dbl> <chr>                        <chr>                <dbl>
## 1 2019     1 Hotel 3 Estrellas      1 Semana                59
## 2 2019     1 Hotel 4 Estrellas      1 Semana                85
## 3 2019     1 Hotel 5 Estrellas      1 Semana               197
## 4 2019     2 Hotel 3 Estrellas      1 Semana                67
## 5 2019     2 Hotel 4 Estrellas      1 Semana                91
## 6 2019     2 Hotel 5 Estrellas      1 Semana               208
## 7 2019     3 Hotel 3 Estrellas      1 Semana                89
## 8 2019     3 Hotel 4 Estrellas      1 Semana               117
## 9 2019     3 Hotel 5 Estrellas      1 Semana               232
## 10 2020     4 Hotel 5 Estrellas      1 Semana                NA
## # i 1 more variable: PRECIO_FIN_SEMANA <dbl>
```

Se limpian los n/a de dataset y se convierte la estructura para hacer el join con dataset de GDP/paro/pernoctaciones/indice_satisfacion

```
# Eliminar todas las filas que tengan NA en las columnas de precios
datos_limpios_hoteles <- datos_hoteles_filtrados_final %>%
  drop_na(PRECIO_ENTRE_SEMANA, PRECIO_FIN_SEMANA)
```

```
# (Opcional) Comprobar cuántas filas te quedan ahora
nrow(datos_limpios_hoteles)
```

```
## [1] 1034
```

```
str(datos_limpios_hoteles)
```

```
## tibble [1,034 x 6] (S3: tbl_df/tbl/data.frame)
## $ AÑO : num [1:1034] 2019 2019 2019 2019 2019 ...
## $ MES : num [1:1034] 1 1 1 1 1 1 1 1 1 2 ...
## $ CATEGORIA_ALOJAMIENTO: chr [1:1034] "Hotel 3 Estrellas" "Hotel 4 Estrellas" "Hotel 5 Estrellas" ...
## $ PERIODO_ANTELACION : chr [1:1034] "1 Mes" "1 Mes" "1 Mes" "2 Semanas" ...
## $ PRECIO_ENTRE_SEMANA : num [1:1034] 58 82 210 58 82 201 57 82 204 63 ...
## $ PRECIO_FIN_SEMANA : num [1:1034] 76 99 228 81 104 235 70 93 219 95 ...
```

```
# 1. Agrupamos primero por si acaso hay duplicados de antelación y promediamos el precio por categoría
hoteles_ancho <- datos_limpios_hoteles %>%
```

```
  group_by(AÑO, MES, CATEGORIA_ALOJAMIENTO) %>%
  summarise(
    Precio_Medio = mean(PRECIO_ENTRE_SEMANA, na.rm = TRUE), # 0 el de fin de semana
    .groups = "drop"
  ) %>%
```

```
# 2. Convertimos las filas de las categorías en columnas independientes
```

```
pivot_wider(
  names_from = CATEGORIA_ALOJAMIENTO,
  values_from = Precio_Medio
)
```

```
# 3. Ahora el left_join es 1 a 1 perfecto y no duplica filas macroeconómicas
```

```
dataset_final_ancho <- dataset_gdp_paro_pernocta_index_final %>%
  left_join(hoteles_ancho, by = c("AÑO", "MES"))
```

```
# Aplicar a dataset final
```

```
dataset_final_ancho <- dataset_final_ancho %>%
  clean_names()
```

```
colnames(dataset_final_ancho)
```

```
## [1] "ano" "mes"
## [3] "inindice_percepcion_turistica_global" "pais_origen"
## [5] "paro" "gdp"
## [7] "total_pernoctaciones" "hotel_3_estrellas"
## [9] "hotel_4_estrellas" "hotel_5_estrellas"
```

```
head(dataset_final_ancha)
```

```
## # A tibble: 6 x 10
##   ano  mes inidice_percepcion_turistica_global pais_origen paro  gdp
##   <dbl> <dbl>                <dbl> <chr>      <dbl>  <dbl>
## 1  2019    1                86.9 Alemania      3  275802.
## 2  2019    1                63.4 Francia      8.8 195065.
## 3  2019    1                87.4 Reino Unido  3.8 233122.
## 4  2019    1                80.0 Suecia      6.3  40372.
## 5  2019    2                76.5 Alemania      3  276228.
## 6  2019    2                78.5 Francia      8.7 195645.
## # i 4 more variables: total_pernoctaciones <dbl>, hotel_3_estrellas <dbl>,
## #   hotel_4_estrellas <dbl>, hotel_5_estrellas <dbl>
```

En base de dataset creado se hará una exploración con el objetivo de identificar alguna correlación significativa entre los variables.

```
dataset_corr <- dataset_final_ancha %>%
  select(total_pernoctaciones, paro, gdp, inidice_percepcion_turistica_global, hotel_3_estrellas, hotel_4_estrellas, hotel_5_estrellas)

# Calculamos la matriz usando Spearman
corr_mat <- cor(dataset_corr, method = "spearman")

# Graficamos
ggpairs(dataset_corr,
  upper = list(continuous = wrap("cor", method = "spearman"))) +
  theme_minimal()
```

```
## Warning: Removed 83 rows containing non-finite outside the scale range
## ('stat_density()').
```

```
## Warning: Removed 83 rows containing missing values
```

```
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
```

```
## Warning: Removed 83 rows containing missing values
```

```
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
```

```
## Warning: Removed 83 rows containing missing values
```

```
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
```

```
## Warning: Removed 83 rows containing missing values
```

```
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
```



```

## Warning: Removed 83 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning: Removed 66 rows containing non-finite outside the scale range
## ('stat_density()').

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning: Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning: Removed 66 rows containing non-finite outside the scale range
## ('stat_density()').

```

```

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 66 rows containing missing values

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning: Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning: Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties
## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values or values outside the scale range
## ('geom_point()').

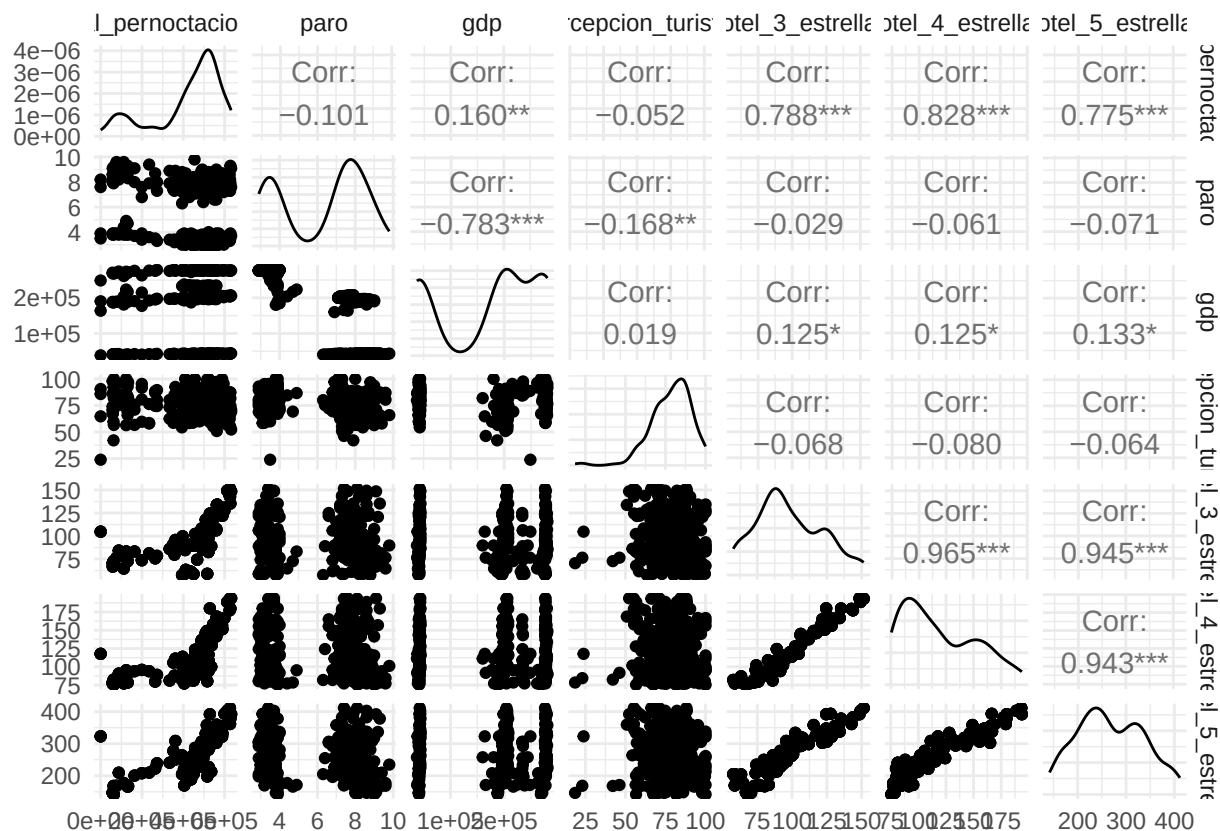
```

```
## Warning: Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning in cor.test.default(x, y, method = method): Cannot compute exact
## p-value with ties

## Warning: Removed 83 rows containing missing values or values outside the scale range
## ('geom_point()').

## Warning: Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 66 rows containing missing values or values outside the scale range
## ('geom_point()').
```



Como podemos observar La columna percepcion_turistica tiene correlaciones muy bajas con el PIB (0.019), con las pernoctaciones (-0.052) y con los precios de los hoteles (alrededor de -0.06) esto nos dice que la opinión o percepción global de los turistas no depende de la macroeconomía ni de si los hoteles están caros o baratos. Su percepción se mantiene estable (alta, según su curva) independientemente de la temporada o la situación del empleo. Tampoco vemos alguna correlacion importante entre GDP en paises de origen, ni nivel de paro.

A primera vista, la teoría económica sugeriría que un aumento en los precios hoteleros debería contraer la demanda y afectar negativamente a las pernoctaciones. Sin embargo, los datos muestran una fuerte correlación positiva ($r \approx 0.80^*$). Esto indica que el mercado turístico analizado está fuertemente movido

por la estacionalidad (oferta y demanda): en los meses de temporada alta, el volumen de pernoctaciones se dispara y, en respuesta a esa alta demanda, los precios hoteleros aumentan simultáneamente, invalidando cualquier efecto contractivo del precio sobre la intención de viaje.

2. Evaluación de la calidad de los datos de dataset resultante

Analizaremos la estructura de dataset. Tenemos que tener en cuenta que sacando datos desde API no hemos limitado hasta que mes/ año y a nosotros nos interesa hasta finales de 2025. Podemos ver unos n/a en paro, GDP, pernoctaciones que tenemos que tratar.

```
summary(dataset_final_ancho)
```

```
##      ano      mes      inidice_percepcion_turistica_global
##  Min.   :2019   Min.    : 1.000   Min.    : 18.57
##  1st Qu.:2020   1st Qu.: 3.000   1st Qu.: 70.98
##  Median :2022   Median : 6.000   Median : 80.17
##  Mean   :2022   Mean    : 6.345   Mean    : 78.71
##  3rd Qu.:2024   3rd Qu.: 9.000   3rd Qu.: 87.30
##  Max.   :2026   Max.    :12.000   Max.    :100.00
##
##  pais_origen      paro      gdp      total_pernoctaciones
##  Length:348      Min.    :2.90   Min.    : 37593   Min.    :    0
##  Class :character 1st Qu.:3.70   1st Qu.: 43384   1st Qu.:393387
##  Mode  :character Median :7.30   Median :201263   Median :487984
##                      Mean   :6.22   Mean   :175657   Mean   :425308
##                      3rd Qu.:8.00   3rd Qu.:275669   3rd Qu.:537924
##                      Max.    :9.80   Max.    :280380   Max.    :631726
##                      NA's    :66    NA's    :66      NA's    :83
##  hotel_3_estrellas hotel_4_estrellas hotel_5_estrellas
##  Min.    : 57.67   Min.    : 75.50   Min.    :141.8
##  1st Qu.: 81.25   1st Qu.: 91.25   1st Qu.:213.0
##  Median : 93.25   Median :111.25   Median :262.0
##  Mean    : 97.75   Mean    :119.57   Mean    :268.1
##  3rd Qu.:117.00   3rd Qu.:146.50   3rd Qu.:323.0
##  Max.    :151.50   Max.    :193.75   Max.    :411.5
##
```

```
# Filtramos para quedarnos solo con los años anteriores a 2026
```

```
dataset_recortado_hasta2025 <- dataset_final_ancho %>%
  filter(ano < 2026)
```

```
# Verificamos el año máximo para estar seguros
```

```
max(dataset_recortado_hasta2025$ano)
```

```
## [1] 2025
```

2.1. Valores Faltantes (NAs) y su tratamiento

```
# Muestra todas las filas que tengan un NA en CUALQUIER columna
```

```
lineas_con_na <- dataset_recortado_hasta2025 %>%
```

```
filter(if_any(everything(), is.na))
```

```
# Para verlas en la consola
print(lineas_con_na)
```

```
## # A tibble: 71 x 10
##   ano   mes indice_percepcion_turistica_global pais_origen paro   gdp
##   <dbl> <dbl>                                <dbl> <chr>   <dbl> <dbl>
## 1 2020     5      88.2 Alemania          3.7 243684.
## 2 2020     5      81.9 Francia           6.9 159910.
## 3 2020     5     100 Reino Unido       3.8 180248.
## 4 2020     5      79.7 Suecia            8.5  37593.
## 5 2020     6      88.6 Alemania          3.8 247870.
## 6 2020     6      46.3 Francia           7.4 165669.
## 7 2020     6      85.4 Reino Unido        4   184837.
## 8 2020     6      86.9 Suecia            9.1  37910.
## 9 2020    10      63.8 Reino Unido       NA      NA
## 10 2020    11      18.6 Reino Unido       NA      NA
## # i 61 more rows
## # i 4 more variables: total_pernotaciones <dbl>, hotel_3_estrellas <dbl>,
## #   hotel_4_estrellas <dbl>, hotel_5_estrellas <dbl>
```

Como se comentó en el apartado anterior, se detectan valores NA en tres de las variables. Empezamos con eliminación de los años de pandemia

```
# Filtramos para quitar el 2020 y 2021
dataset_sin_pandemia <- dataset_recortado_hasta2025 %>%
  filter(!ano %in% c(2020, 2021))

# Si quieres verificar qué años te quedaron:
unique(dataset_sin_pandemia$ano)
```

```
## [1] 2019 2022 2023 2024 2025
```

```
write_csv(dataset_sin_pandemia, "dataset_economico_final_sin_pandemia.csv")
```

```
# Muestra todas las filas que tengan un NA en CUALQUIER columna
lineas_con_na_sin_pandemia <- dataset_sin_pandemia %>%
  filter(if_any(everything(), is.na))

# Para verlas en la consola
print(lineas_con_na_sin_pandemia)
```

```
## # A tibble: 48 x 10
##   ano   mes indice_percepcion_turistica_global pais_origen paro   gdp
##   <dbl> <dbl>                                <dbl> <chr>   <dbl> <dbl>
## 1 2022     1      86.8 Reino Unido        NA      NA
## 2 2022     2      85.8 Reino Unido        NA      NA
## 3 2022     3      81.7 Reino Unido        NA      NA
## 4 2022     4      87.4 Reino Unido        NA      NA
## 5 2022     5      82.2 Reino Unido        NA      NA
```

```
## 6 2022      6      84.8 Reino Unido      NA      NA
## 7 2022      7      90.6 Reino Unido      NA      NA
## 8 2022      8      85.8 Reino Unido      NA      NA
## 9 2022      9      83.7 Reino Unido      NA      NA
## 10 2022     10      83.0 Reino Unido      NA      NA
## # i 38 more rows
## # i 4 more variables: total_pernoctaciones <dbl>, hotel_3_estrellas <dbl>,
## #   hotel_4_estrellas <dbl>, hotel_5_estrellas <dbl>
```

Aun así existen datos n/a de Reino Unido por salida de Eurozona y que Eurostat dejó de publicarlos. Vamos a usar otra fuente de datos para estadísticas de Reino Unido - Banco Mundial. Después se aplicará de nuevo el método de Denton-Cholette para aumentar la frecuencia de datos (GDP, paro) de trimestral a mensual.

```
message("Cargando dataset original...")
```

```
## Cargando dataset original...
```

```
df_original <- read_csv("dataset_economico_final_sin_pandemia.csv")
```

```
## Rows: 240 Columns: 10
```

```
## -- Column specification -----
## Delimiter: ","
## chr (1): pais_origen
## dbl (9): ano, mes, inidice_percepcion_turistica_global, paro, gdp, total_per...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# 2. BLOQUE DE DATOS REALES Y OFICIALES DEL BANCO MUNDIAL (UK)
# Introducidos directamente para evitar fallos de conexión o APIs de terceros.
# PIB en dólares corrientes y tasa de paro media anual.
uk_gdp_anual <- tibble(
  ano = c(2022, 2023, 2024, 2025),
  Paro_Mensual = c(3.72, 4.00, 4.27, 4.30), # Datos macro reales
  GDP_Anual = c(3084128000000, 3340032000000, 3450210000000, 3501900000000) # USD reales
)
```

```
# 3. Aplicar desagregación temporal Denton-Cholette mensual
message("Aplicando método Denton-Cholette para generar los 48 meses...")
```

```
## Aplicando método Denton-Cholette para generar los 48 meses...
```

```
# Convertimos el GDP a millones de dólares (escala de 6 dígitos para que cuadre con tu 2019)
gdp_anual_ts <- ts(uk_gdp_anual$GDP_Anual / 1000000, start = 2022, frequency = 1)
```

```
# Desagregación mensual exacta (Genera los 48 meses perfectos)
gdp_mensual_desagregado <- td(gdp_anual_ts ~ 1, conversion = "sum", to = "monthly", method = "denton-cholette")
uk_gdp_mensual_valores <- as.numeric(predict(gdp_mensual_desagregado))
```

```
# 4. Crear el parche de datos con los nombres EXACTOS de tus columnas
uk_parche <- tibble(
  ano = rep(2022:2025, each = 12),
  mes = rep(1:12, times = 4),
  pais_origen = "Reino Unido",
  paro_nuevo = rep(uk_gdp_anual$Paro_Mensual, each = 12),
  gdp_nuevo = uk_gdp_mensual_valores
)
```

```
# 5. Fusionar el dataset
message("Inyectando los nuevos datos en el archivo...")
```

```
## Inyectando los nuevos datos en el archivo...
```

```
dataset_final_corregido <- df_original %>%
  left_join(uk_parche, by = c("ano", "mes", "pais_origen")) %>%
  mutate(
    # Reemplazamos los NA de Reino Unido usando variables nativas
    paro = if_else(pais_origen == "Reino Unido" & is.na(paro), paro_nuevo, paro),
    gdp = if_else(pais_origen == "Reino Unido" & is.na(gdp), gdp_nuevo, gdp)
  ) %>%
  # Limpiamos las columnas temporales del parche
  select(-paro_nuevo, -gdp_nuevo)

# 6. Guardar y verificar el resultado
write_csv(dataset_final_corregido, "dataset_economico_final_arreglado.csv")

# Comprobación rápida en consola para que veas que YA NO ESTÁ VACÍO
message("\n--- COMPROBACIÓN DE DATOS RELLENADOS (REINO UNIDO 2022) ---")
```

```
##
## --- COMPROBACIÓN DE DATOS RELLENADOS (REINO UNIDO 2022) ---
```

```
dataset_final_corregido %>%
  filter(pais_origen == "Reino Unido" & ano == 2022) %>%
  select(ano, mes, pais_origen, paro, gdp) %>%
  print(n = 12)
```

```
## # A tibble: 12 x 5
##   ano  mes pais_origen  paro    gdp
##   <dbl> <dbl> <chr>         <dbl>  <dbl>
## 1  2022     1 Reino Unido  3.72 251933.
## 2  2022     2 Reino Unido  3.72 252146.
## 3  2022     3 Reino Unido  3.72 252572.
## 4  2022     4 Reino Unido  3.72 253211.
## 5  2022     5 Reino Unido  3.72 254063.
## 6  2022     6 Reino Unido  3.72 255129.
## 7  2022     7 Reino Unido  3.72 256407.
## 8  2022     8 Reino Unido  3.72 257898.
## 9  2022     9 Reino Unido  3.72 259603.
```

```
## 10 2022    10 Reino Unido  3.72 261520.
## 11 2022    11 Reino Unido  3.72 263651.
## 12 2022    12 Reino Unido  3.72 265995.
```

Como podemos comprobar hemos eliminado los n/a de GDP y paro y nos quedan aún en pernoctaciones.

```
dataset_final_con_uk <- read_csv("dataset_economico_final_arreglado.csv")
```

```
## Rows: 240 Columns: 10
## -- Column specification -----
## Delimiter: ","
## chr (1): pais_origen
## dbl (9): ano, mes, inidice_percepcion_turistica_global, paro, gdp, total_per...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
summary(dataset_final_con_uk)
```

```
##      ano      mes      inidice_percepcion_turistica_global
## Min.   :2019   Min.   : 1.00   Min.   : 50.87
## 1st Qu.:2022   1st Qu.: 3.75   1st Qu.: 71.53
## Median :2023   Median : 6.50   Median : 80.66
## Mean   :2023   Mean   : 6.50   Mean   : 79.66
## 3rd Qu.:2024   3rd Qu.: 9.25   3rd Qu.: 87.47
## Max.   :2025   Max.   :12.00   Max.   :100.00
##
## pais_origen      paro      gdp      total_pernoctaciones
## Length:240      Min.   :2.900   Min.   : 40372   Min.   :272681
## Class :character 1st Qu.:3.700   1st Qu.:157037   1st Qu.:459618
## Mode  :character Median :5.300   Median :220283   Median :516112
##                      Mean  :5.686   Mean  :197883   Mean  :509257
##                      3rd Qu.:7.500   3rd Qu.:277258   3rd Qu.:559653
##                      Max.   :9.800   Max.   :292693   Max.   :631726
##                      NA's   :48
## hotel_3_estrellas hotel_4_estrellas hotel_5_estrellas
## Min.   : 57.67   Min.   : 80.25   Min.   :196.2
## 1st Qu.: 89.00   1st Qu.:101.25   1st Qu.:236.6
## Median :103.75   Median :126.88   Median :293.5
## Mean   :104.69   Mean   :130.09   Mean   :290.0
## 3rd Qu.:124.50   3rd Qu.:156.69   3rd Qu.:333.4
## Max.   :151.50   Max.   :193.75   Max.   :411.5
##
```

Para tratar los datos n/a de pernoctaciones se utilizará un modelo estadístico de “Cuota de Mercado”. En economía del turismo, la demanda hacia un destino (España) suele mantener proporciones estables entre mercados emisores maduros. Si en 2019 los turistas británicos representaban, por ejemplo, el 40% del total de pernoctaciones podemos asumir que su “cuota de mercado” se mantiene relativamente estable o vinculada a su poder adquisitivo (gdp). Podemos crear un modelo que calcule la relación histórica de 2019 entre las pernoctaciones de Reino Unido y las de Alemania o Francia, ponderada por su PIB, y proyectar el volumen de 2022-2025


```

df_gdp_ok <- read_csv("dataset_economico_final_arreglado.csv")

## Rows: 240 Columns: 10
## -- Column specification -----
## Delimiter: ","
## chr (1): pais_origen
## dbl (9): ano, mes, inidice_percepcion_turistica_global, paro, gdp, total_per...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

# 1. Calcular el total de pernoctaciones de los otros países mes a mes
df_otros_paises <- df_gdp_ok %>%
  filter(pais_origen != "Reino Unido") %>%
  group_by(ano, mes) %>%
  summarise(pernoctaciones_resto = sum(total_pernoctaciones, na.rm = TRUE), .groups = 'drop')

# 2. Unir esta métrica al dataset global
df_con_resto <- df_gdp_ok %>%
  left_join(df_otros_paises, by = c("ano", "mes"))

# 3. Entrenar un modelo con los datos reales de UK en 2019
# Explicamos las pernoctaciones de UK en función del resto de Europa y de su propio GDP mensual
modelo_cuota <- lm(
  total_pernoctaciones ~ pernoctaciones_resto + gdp + inidice_percepcion_turistica_global,
  data = df_con_resto %>% filter(pais_origen == "Reino Unido" & ano == 2019)
)

# 4. Predecir y rellenar los NA para 2022-2025
df_final_cuota <- df_con_resto %>%
  mutate(
    prediccion = predict(modelo_cuota, newdata = .),
    total_pernoctaciones = if_else(
      pais_origen == "Reino Unido" & is.na(total_pernoctaciones),
      round(prediccion, 0),
      total_pernoctaciones
    )
  ) %>%
  select(-pernoctaciones_resto, -prediccion)

# Guardar el archivo
write_csv(df_final_cuota, "dataset_economico_final_completo_a_unir.csv")

data_ultima <- read_csv("dataset_economico_final_completo_a_unir.csv")

## Rows: 240 Columns: 10
## -- Column specification -----
## Delimiter: ","
## chr (1): pais_origen
## dbl (9): ano, mes, inidice_percepcion_turistica_global, paro, gdp, total_per...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

```

```
summary(data_ultima)
```

```
##      ano      mes      inidice_percepcion_turistica_global
## Min.   :2019   Min.   : 1.00   Min.   : 50.87
## 1st Qu.:2022   1st Qu.: 3.75   1st Qu.: 71.53
## Median :2023   Median : 6.50   Median : 80.66
## Mean   :2023   Mean   : 6.50   Mean   : 79.66
## 3rd Qu.:2024   3rd Qu.: 9.25   3rd Qu.: 87.47
## Max.   :2025   Max.   :12.00   Max.   :100.00
## pais_origen      paro      gdp      total_pernoctaciones
## Length:240      Min.   :2.900   Min.   : 40372   Min.   :272681
## Class :character 1st Qu.:3.700   1st Qu.:157037   1st Qu.:465183
## Mode  :character Median :5.300   Median :220283   Median :518982
##                Mean   :5.686   Mean   :197883   Mean   :510502
##                3rd Qu.:7.500   3rd Qu.:277258   3rd Qu.:559667
##                Max.   :9.800   Max.   :292693   Max.   :631726
## hotel_3_estrellas hotel_4_estrellas hotel_5_estrellas
## Min.   : 57.67   Min.   : 80.25   Min.   :196.2
## 1st Qu.: 89.00   1st Qu.:101.25   1st Qu.:236.6
## Median :103.75   Median :126.88   Median :293.5
## Mean   :104.69   Mean   :130.09   Mean   :290.0
## 3rd Qu.:124.50   3rd Qu.:156.69   3rd Qu.:333.4
## Max.   :151.50   Max.   :193.75   Max.   :411.5
```

2.2. Analisis completo de datasat resultante para valores ausentes, ceros etc

```
df_evaluar <- read_csv("dataset_economico_final_completo_a_unir.csv", show_col_types = FALSE)

# --- MÉTODO 1: REPORTE GENERAL POR COLUMNA ---
message("=== AUDITORÍA DE ANOMALÍAS POR COLUMNA ===")
```

```
## === AUDITORÍA DE ANOMALÍAS POR COLUMNA ===
```

```
reporte_anomalias <- df_evaluar %>%
  # Pasamos a formato largo para analizar todas las variables numéricas a la vez
  pivot_longer(
    cols = where(is.numeric),
    names_to = "Variable",
    values_to = "Valor"
  ) %>%
  group_by(Variable) %>%
  summarise(
    Total_Registros = n(),
    Total_NA        = sum(is.na(Valor)),
    Total_Ceros     = sum(Valor == 0, na.rm = TRUE),
    Total_Negativos = sum(Valor < 0, na.rm = TRUE),
    .groups = "drop"
  )

print(reporte_anomalias)
```

```
## # A tibble: 9 x 5
##   Variable                Total_Registros Total_NA Total_Ceros Total_Negativos
##   <chr>                  <int>      <int>      <int>      <int>
## 1 ano                    240          0          0          0
## 2 gdp                    240          0          0          0
## 3 hotel_3_estrellas      240          0          0          0
## 4 hotel_4_estrellas      240          0          0          0
## 5 hotel_5_estrellas      240          0          0          0
## 6 inidice_percepcion_turis~ 240          0          0          0
## 7 mes                    240          0          0          0
## 8 paro                    240          0          0          0
## 9 total_pernoctaciones    240          0          0          0
```

```
# --- MÉTODO 2: FILTRAR LAS FILAS EXACTAS QUE TIENEN ERRORES ---
# Si quieres ver exactamente qué filas o meses específicos tienen un problema
message("\n=== BUSCANDO FILAS CON VALORES CRÍTICOS (NA, Negativos o Ceros) ===")
```

```
##
## === BUSCANDO FILAS CON VALORES CRÍTICOS (NA, Negativos o Ceros) ===
```

```
filas_sospechosas <- df_evaluar %>%
  filter(
    # Verifica si hay algún NA en todo el registro
    if_any(where(is.numeric), is.na) |
    # Verifica si el GDP o las Pernoctaciones son menores o iguales a cero
    gdp <= 0 |
    total_pernoctaciones <= 0 |
    paro < 0
  )

if (nrow(filas_sospechosas) == 0) {
  message("¡Excelente! No se encontraron valores NA, negativos en variables críticas ni ceros extraños.")
} else {
  message(paste("Se encontraron", nrow(filas_sospechosas), "filas con anomalías. Aquí tienes las primeras:"))
  print(head(filas_sospechosas))
}
```

```
## ¡Excelente! No se encontraron valores NA, negativos en variables críticas ni ceros extraños.
```

Fuente de datos 4: festivos y eventos

1. Carga y estructura de los datos

1.1 Carga de los datos

Para el análisis se utilizó un conjunto de datos correspondiente a las pernoctaciones turísticas en la provincia de Sevilla durante el período de estudio. Los datos fueron importados desde un archivo .csv y posteriormente procesados en R para su análisis exploratorio y modelado.

Los datos fueron importados desde un archivo .csv y posteriormente procesados en R para su análisis exploratorio y modelado. Adicionalmente, se incorporaron manualmente tres variables explicativas relacionadas con eventos y festividades relevantes en Sevilla:

- **holiday_count:** Número de días festivos presentes en cada período analizado.
- **holy_week:** Variable binaria que indica la presencia de la Semana Santa.
- **feria_abril:** Variable binaria que indica la celebración de la Feria de Abril.

Estas variables fueron construidas manualmente a partir del calendario oficial de festividades de la provincia de Sevilla, con el objetivo de capturar el posible impacto de eventos turísticos y festivos sobre el comportamiento de las pernoctaciones.

La incorporación de estas variables permite enriquecer el conjunto de datos original y considerar factores estacionales específicos del contexto turístico de Sevilla.

```
eventos_festivos <- Pernoctaciones_2016_2025_total <- read_delim("https://raw.githubusercontent.com/adr/
  delim = ";", escape_double = FALSE, trim_ws = TRUE)
```

```
## Rows: 120 Columns: 5
## -- Column specification -----
## Delimiter: ";"
## chr (1): Fecha
## dbl (4): holiday_count, holy_week, feria, Pernoctaciones_Totales
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Los años 2020 y 2021 fueron excluidos del análisis debido al impacto excepcional de la pandemia de COVID-19 sobre la actividad turística. La eliminación de estos períodos permite evitar distorsiones en el comportamiento de la serie y mejorar la representatividad de los patrones turísticos habituales.

```
#Se convierten todos los nombres de las columnas a minúsculas
names(eventos_festivos) <- tolower(names(eventos_festivos))
head(eventos_festivos)
```

```
## # A tibble: 6 x 5
##   fecha    holiday_count holy_week feria pernoctaciones_totales
##   <chr>          <dbl>     <dbl> <dbl>          <dbl>
## 1 2025-12             3         0     0          522493
## 2 2025-11             1         0     0          560063
## 3 2025-10             1         0     0          628123
## 4 2025-09             0         0     0          571732
## 5 2025-08             1         0     0          523211
## 6 2025-07             0         0     0          487984
```

```
#Cambiando el tipo de la variable fecha a Date
df_eventos_festivos <- eventos_festivos %>%
  mutate(fecha_objetivo = ymd(paste0(fecha, "-01")))

# Eliminar todas las filas que pertenezcan al año 2020 y 2021 completo
eventos_limpio <- df_eventos_festivos %>%
  filter(!format(fecha_objetivo, "%Y") %in% c("2020", "2021"))
```

1.2 Estructura

Se evaluó la estructura de los datos para detectar posibles anomalías en los formatos, tipos de datos incorrectos, etc.

```
# Vemos las primeras filas y la estructura.  
head(eventos_limpio)
```

```
## # A tibble: 6 x 6  
##   fecha    holiday_count holy_week feria pernoctaciones_totales fecha_objetivo  
##   <chr>          <dbl>      <dbl> <dbl>          <dbl> <date>  
## 1 2025-12           3          0     0          522493 2025-12-01  
## 2 2025-11           1          0     0          560063 2025-11-01  
## 3 2025-10           1          0     0          628123 2025-10-01  
## 4 2025-09           0          0     0          571732 2025-09-01  
## 5 2025-08           1          0     0          523211 2025-08-01  
## 6 2025-07           0          0     0          487984 2025-07-01
```

```
str(eventos_limpio)
```

```
## tibble [96 x 6] (S3: tbl_df/tbl/data.frame)  
## $ fecha          : chr [1:96] "2025-12" "2025-11" "2025-10" "2025-09" ...  
## $ holiday_count  : num [1:96] 3 1 1 0 1 0 0 2 2 2 ...  
## $ holy_week      : num [1:96] 0 0 0 0 0 0 0 0 1 0 ...  
## $ feria          : num [1:96] 0 0 0 0 0 0 0 1 0 0 ...  
## $ pernoctaciones_totales: num [1:96] 522493 560063 628123 571732 523211 ...  
## $ fecha_objetivo : Date[1:96], format: "2025-12-01" "2025-11-01" ...
```

Tras la revisión de la estructura del conjunto de datos, no se observaron anomalías en los tipos de variables ni inconsistencias en la información cargada. Asimismo, el conjunto de datos no presentó valores faltantes (NA), por lo que no fue necesario aplicar técnicas adicionales de imputación o tratamiento de datos incompletos antes de continuar con el análisis.

Creación de dataset maestro

GDP/paro/pernoctaciones/indice_percepcion/precio_hoteles con dataset de clima, festivos y pivoteo de dataset final.

```
datos_clima <- read_csv("Dataset_Maestro_Clima_sinCOVID.csv")
```

```
## Rows: 120 Columns: 7  
## -- Column specification -----  
## Delimiter: ","  
## chr   (1): fecha  
## dbl   (5): tm_max, tm_min, inso, dias_lluvia, pernoctaciones_totales  
## date  (1): fecha_objetivo  
##  
## i Use 'spec()' to retrieve the full column specification for this data.  
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
summary(datos_clima)
```

```
##      fecha          tm_max      tm_min      inso
## Length:120      Min.    :15.00  Min.    : 4.00  Min.    : 4.500
## Class :character 1st Qu.:19.00  1st Qu.: 9.05  1st Qu.: 7.075
## Mode  :character Median :25.65  Median :12.70 Median : 9.000
##                Mean  :26.26  Mean  :13.39 Mean   : 9.091
##                3rd Qu.:32.75  3rd Qu.:18.02 3rd Qu.:11.167
##                Max.   :39.40  Max.   :22.80 Max.   :13.300
## dias_lluvia      pernoctaciones_totales fecha_objetivo
## Min.    : 0.000  Min.    :    0      Min.    :2016-01-01
## 1st Qu.: 1.000  1st Qu.:383467      1st Qu.:2018-06-23
## Median : 3.000  Median :460734      Median :2020-12-16
## Mean   : 3.975  Mean   :424053      Mean   :2020-12-15
## 3rd Qu.: 6.000  3rd Qu.:522557      3rd Qu.:2023-06-08
## Max.   :18.000  Max.   :631726      Max.   :2025-12-01
```

```
message("Cargando ambos datasets...")
```

```
## Cargando ambos datasets...
```

```
# Reemplaza el nombre del archivo por el tuyo actual si varía (ej: dataset_final_completo_cuota.csv)
df_economico <- read_csv("dataset_economico_final_completo_a_unir.csv", show_col_types = FALSE)
df_clima      <- read_csv("Dataset_Maestro_Clima_sinCOVID.csv", show_col_types = FALSE)

# --- PASO 2: CORRECCIÓN Y PREPARACIÓN DEL DATASET CLIMÁTICO ---
message("Transformando fechas y limpiando el dataset climático...")
```

```
## Transformando fechas y limpiando el dataset climático...
```

```
df_clima_limpio <- df_clima %>%
  mutate(
    # Convertimos la columna fecha_objetivo a un formato de fecha real de R
    Fecha_Parsed = as.Date(fecha_objetivo),
    # Extraemos el año y el mes como números enteros para que coincidan con tus columnas
    ano = as.numeric(format(Fecha_Parsed, "%Y")),
    mes = as.numeric(format(Fecha_Parsed, "%m"))
  ) %>%
  # Eliminamos variables que causarían conflicto o que no necesitas:
  # - 'fecha' y 'fecha_objetivo' (ya tenemos ano y mes)
  # - 'pernoctaciones_totales' (para no duplicar con tus pernoctaciones por país)
  select(ano, mes, tm_max, tm_min, inso, dias_lluvia)

# --- PASO 3: UNIÓN (JOIN) DE AMBOS ARCHIVOS ---
message("Realizando la unión de datos macroeconómicos y climáticos...")
```

```
## Realizando la unión de datos macroeconómicos y climáticos...
```

```
# Usamos left_join para preservar la estructura exacta de dataset de países
dataset_final_fusionado <- df_economico %>%
  left_join(df_clima_limpio, by = c("ano", "mes"))
```

```
# --- PASO 4: AUDITORÍA DE CONTROL DE CALIDAD ---
message("\n=== VERIFICACIÓN POST-JOIN ===")
```

```
##
## === VERIFICACIÓN POST-JOIN ===
```

```
# Contamos si ha quedado algún vacío en las nuevas variables climáticas
nas_detectados <- sum(is.na(dataset_final_fusionado$tm_max))
```

```
if(nas_detectados == 0) {
  message("¡Perfecto! Todas las filas económicas han recibido sus datos de clima correspondientes.")
} else {
  warning(paste("Atención: Hay", nas_detectados, "filas sin datos de clima. Revisa los rangos de años."))
}
```

```
## ¡Perfecto! Todas las filas económicas han recibido sus datos de clima correspondientes.
```

```
# --- PASO 5: GUARDAR EL ARCHIVO MAESTRO FINAL ---
```

```
write_csv(dataset_final_fusionado, "dataset_final_completo_economico_clima.csv")
message("¡Proceso completado con éxito! Archivo maestro 'dataset_final_completo_economico_clima.csv' guardado")
```

```
## ¡Proceso completado con éxito! Archivo maestro 'dataset_final_completo_economico_clima.csv' guardado
```

```
# Vista previa para comprobar el resultado estructural
print(head(dataset_final_fusionado, 8))
```

```
## # A tibble: 8 x 14
##   ano mes inidice_percepcion_turistica_global pais_origen paro gdp
##   <dbl> <dbl>                                <dbl> <chr>      <dbl> <dbl>
## 1 2019 1 86.9 Alemania 3 275802.
## 2 2019 1 63.4 Francia 8.8 195065.
## 3 2019 1 87.4 Reino Unido 3.8 233122.
## 4 2019 1 80.0 Suecia 6.3 40372.
## 5 2019 2 76.5 Alemania 3 276228.
## 6 2019 2 78.5 Francia 8.7 195645.
## 7 2019 2 94.0 Reino Unido 3.7 233513.
## 8 2019 2 88.8 Suecia 6.9 40436.
## # i 8 more variables: total_pernoctaciones <dbl>, hotel_3_estrellas <dbl>,
## # hotel_4_estrellas <dbl>, hotel_5_estrellas <dbl>, tm_max <dbl>,
## # tm_min <dbl>, inso <dbl>, dias_lluvia <dbl>
```

```
datos_festivos <- read_delim("eventos_festivos_Sevilla.csv",
  delim = ";", escape_double = FALSE, trim_ws = TRUE)
```

```
## Rows: 120 Columns: 4
```

```
## -- Column specification -----
## Delimiter: ";"
## chr (1): Fecha
## dbl (3): holiday_count, holy_week, feria
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(datos_festivos)
```

```
## # A tibble: 6 x 4
##   Fecha    holiday_count holy_week feria
##   <chr>          <dbl>      <dbl> <dbl>
## 1 2025-12           3          0      0
## 2 2025-11           1          0      0
## 3 2025-10           1          0      0
## 4 2025-09           0          0      0
## 5 2025-08           1          0      0
## 6 2025-07           0          0      0
```

```
message("Cargando dataset maestro y el archivo de eventos...")
```

```
## Cargando dataset maestro y el archivo de eventos...
```

```
df_maestro <- read_csv("dataset_final_completo_economico_clima.csv", show_col_types = FALSE)
df_eventos <- eventos_limpio
```

```
# --- PASO 2: LIMPIEZA CONSERVANDO EL TOTAL GLOBAL ---
```

```
df_eventos_limpio <- df_eventos %>%
  mutate(
    Fecha_Parsed = as.Date(fecha_objetivo),
    ano = as.numeric(format(Fecha_Parsed, "%Y")),
    mes = as.numeric(format(Fecha_Parsed, "%m"))
  ) %>%
  # CONSERVAMOS EL DATO:
  select(
    ano,
    mes,
    holiday_count,
    holy_week,
    feria
  ) %>%
  distinct(ano, mes, .keep_all = TRUE)
```

```
# --- PASO 3: UNIÓN FINAL ---
```

```
dataset_final_completo <- df_maestro %>%
  left_join(df_eventos_limpio, by = c("ano", "mes"))
```

```
# Rellenar posibles vacíos en eventos con 0
```

```
dataset_final_completo <- dataset_final_completo %>%
  mutate(
    holiday_count = replace_na(holiday_count, 0),
```



```

    holy_week      = replace_na(holy_week, 0),
    feria          = replace_na(feria, 0)
  )

# --- PASO 4: GUARDAR EL DATASET COMPLETO ---
write_csv(dataset_final_completo, "dataset_final_completo_economico_clima_eventos.csv")
message("¡Perfecto! Dataset guardado conservando la variable global como 'pax_total_destino_espana'.")

```

```
## ¡Perfecto! Dataset guardado conservando la variable global como 'pax_total_destino_espana'.
```

```
df <- read_csv("dataset_final_completo_economico_clima_eventos.csv")
```

```
## Rows: 240 Columns: 17
## -- Column specification -----
## Delimiter: ","
## chr (1): pais_origen
## dbl (16): ano, mes, inidice_percepcion_turistica_global, paro, gdp, total_pe...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
summary(df)
```

```
##      ano      mes      inidice_percepcion_turistica_global
## Min.   :2019   Min.   : 1.00   Min.   : 50.87
## 1st Qu.:2022   1st Qu.: 3.75   1st Qu.: 71.53
## Median :2023   Median : 6.50   Median : 80.66
## Mean   :2023   Mean   : 6.50   Mean   : 79.66
## 3rd Qu.:2024   3rd Qu.: 9.25   3rd Qu.: 87.47
## Max.   :2025   Max.   :12.00   Max.   :100.00
## pais_origen      paro      gdp      total_pernoctaciones
## Length:240      Min.   :2.900   Min.   : 40372   Min.   :272681
## Class :character 1st Qu.:3.700   1st Qu.:157037   1st Qu.:465183
## Mode  :character Median :5.300   Median :220283   Median :518982
##                      Mean  :5.686   Mean  :197883   Mean  :510502
##                      3rd Qu.:7.500   3rd Qu.:277258   3rd Qu.:559667
##                      Max.   :9.800   Max.   :292693   Max.   :631726
## hotel_3_estrellas hotel_4_estrellas hotel_5_estrellas      tm_max
## Min.   : 57.67   Min.   : 80.25   Min.   :196.2   Min.   :15.60
## 1st Qu.: 89.00   1st Qu.:101.25   1st Qu.:236.6   1st Qu.:19.15
## Median :103.75   Median :126.88   Median :293.5   Median :26.25
## Mean   :104.69   Mean   :130.09   Mean   :290.0   Mean   :26.46
## 3rd Qu.:124.50   3rd Qu.:156.69   3rd Qu.:333.4   3rd Qu.:31.77
## Max.   :151.50   Max.   :193.75   Max.   :411.5   Max.   :39.40
##      tm_min      inso      dias_lluvia      holiday_count
## Min.   : 4.20   Min.   : 4.500   Min.   : 0.000   Min.   :0.00
## 1st Qu.: 9.25   1st Qu.: 6.900   1st Qu.: 0.750   1st Qu.:0.75
## Median :12.50   Median : 9.050   Median : 3.000   Median :1.00
## Mean   :13.35   Mean   : 9.024   Mean   : 3.883   Mean   :1.25
## 3rd Qu.:17.93   3rd Qu.:11.175   3rd Qu.: 6.000   3rd Qu.:2.00
## Max.   :21.90   Max.   :13.300   Max.   :18.000   Max.   :3.00
```

```
##      holy_week      feria
## Min.      :0.00000 Min.      :0.00000
## 1st Qu.:0.00000 1st Qu.:0.00000
## Median :0.00000 Median :0.00000
## Mean    :0.08333 Mean    :0.08333
## 3rd Qu.:0.00000 3rd Qu.:0.00000
## Max.    :1.00000 Max.    :1.00000
```

```
message("Cargando dataset maestro fusionado...")
```

```
## Cargando dataset maestro fusionado...
```

```
df_maestro <- read_csv("dataset_final_completo_economico_clima_eventos.csv", show_col_types = FALSE)
```

```
# --- PASO 2: PIVOTADO ANCHO (WIDE FORMAT) ---
```

```
message("Pivotando los países para consolidar una única fila por mes...")
```

```
## Pivotando los países para consolidar una única fila por mes...
```

```
df_mensual_unico_economico_clima_eventos <- df_maestro %>%
  # pivot_wider se encarga de reestructurar la tabla
  pivot_wider(
    # El año y el mes (junto con las variables climáticas, de eventos y hoteles que son comunes)
    # se quedan intactos identificando a la fila única.
    id_cols = c(ano, mes, hotel_3_estrellas, hotel_4_estrellas, hotel_5_estrellas,
                tm_max, tm_min, inso, dias_lluvia, holiday_count, holy_week, feria),

    # ¿Qué columna define las nuevas categorías de las columnas?
    names_from = pais_origen,

    # ¿Qué valores numéricos van a rellenar esas nuevas columnas?
    values_from = c(gdp, paro, total_pernoctaciones, indice_percepcion_turistica_global),

    # Configura cómo se llamarán las nuevas columnas (Ej: gdp_Alemania, paro_Francia)
    names_sep = "_"
  ) %>%
  # Ordenamos cronológicamente para que la serie temporal tenga sentido lineal
  arrange(ano, mes)
```

```
# --- PASO 3: AUDITORÍA DE ESTRUCTURA ---
```

```
message("\n=== COMPROBACIÓN DEL NUEVO DATASET ANCHO ===")
```

```
##
```

```
## === COMPROBACIÓN DEL NUEVO DATASET ANCHO ===
```

```
print(paste("Registros iniciales (filas por país):", nrow(df_maestro)))
```

```
## [1] "Registros iniciales (filas por país): 240"
```

```

print(paste("Registros finales (filas únicas por mes):", nrow(df_mensual_unico_economico_clima_eventos))

## [1] "Registros finales (filas únicas por mes): 60"

print(paste("Número total de columnas predictoras:", ncol(df_mensual_unico_economico_clima_eventos)))

## [1] "Número total de columnas predictoras: 28"

# --- PASO 4: GUARDAR EL DATASET GLOBAL ---
write_csv(df_mensual_unico_economico_clima_eventos, "dataset_mensual_unico_economico_clima_eventos.csv")
message("\n¡Hecho! El archivo 'dataset_mensual_formato_ancho.csv' está listo para modelos globales.")

##
## ¡Hecho! El archivo 'dataset_mensual_formato_ancho.csv' está listo para modelos globales.

# Mostrar una vista previa de las nuevas columnas generadas
print(colnames(df_mensual_unico_economico_clima_eventos))

## [1] "ano"
## [2] "mes"
## [3] "hotel_3_estrellas"
## [4] "hotel_4_estrellas"
## [5] "hotel_5_estrellas"
## [6] "tm_max"
## [7] "tm_min"
## [8] "inso"
## [9] "dias_lluvia"
## [10] "holiday_count"
## [11] "holy_week"
## [12] "feria"
## [13] "gdp_Alemania"
## [14] "gdp_Francia"
## [15] "gdp_Reino Unido"
## [16] "gdp_Suecia"
## [17] "paro_Alemania"
## [18] "paro_Francia"
## [19] "paro_Reino Unido"
## [20] "paro_Suecia"
## [21] "total_pernoctaciones_Alemania"
## [22] "total_pernoctaciones_Francia"
## [23] "total_pernoctaciones_Reino Unido"
## [24] "total_pernoctaciones_Suecia"
## [25] "inindice_percepcion_turistica_global_Alemania"
## [26] "inindice_percepcion_turistica_global_Francia"
## [27] "inindice_percepcion_turistica_global_Reino Unido"
## [28] "inindice_percepcion_turistica_global_Suecia"

head(df_mensual_unico_economico_clima_eventos)

```

```
## # A tibble: 6 x 28
##   ano  mes hotel_3_estrellas hotel_4_estrellas hotel_5_estrellas tm_max
##   <dbl> <dbl>           <dbl>           <dbl>           <dbl> <dbl>
## 1 2019   1             57.7             82             205    17
## 2 2019   2             63.3             87             204.   20.4
## 3 2019   3             82.7            109             222    23
## 4 2019   4             125            156.             306.   22.5
## 5 2019   5             122.            165             303.   30.9
## 6 2019   6              89            124.             255   31.5
## # i 22 more variables: tm_min <dbl>, inso <dbl>, dias_lluvia <dbl>,
## #   holiday_count <dbl>, holy_week <dbl>, feria <dbl>, gdp_Alemania <dbl>,
## #   gdp_Francia <dbl>, 'gdp_Reino Unido' <dbl>, gdp_Suecia <dbl>,
## #   paro_Alemania <dbl>, paro_Francia <dbl>, 'paro_Reino Unido' <dbl>,
## #   paro_Suecia <dbl>, total_pernoctaciones_Alemania <dbl>,
## #   total_pernoctaciones_Francia <dbl>,
## #   'total_pernoctaciones_Reino Unido' <dbl>, ...
```